

Autoencoders

Agenda	Unsupervised Learning
	Autoencoders
	Convolutional Autoencoder
	Transpose Convolution
	Semi-Supervised Learning

Slide Attribution

These slides contain materials from various sources. Special thanks to the following authors:

- Lisa Zhang
- Justin Johnson

Last Time

- CNN Architectures
- Transfer Learning
- CNN Limitations
- Feature Visualization
- Style Transfer
- Object Detection

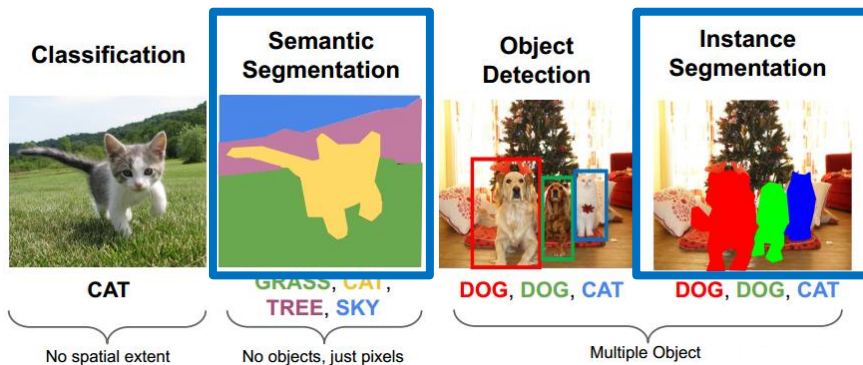
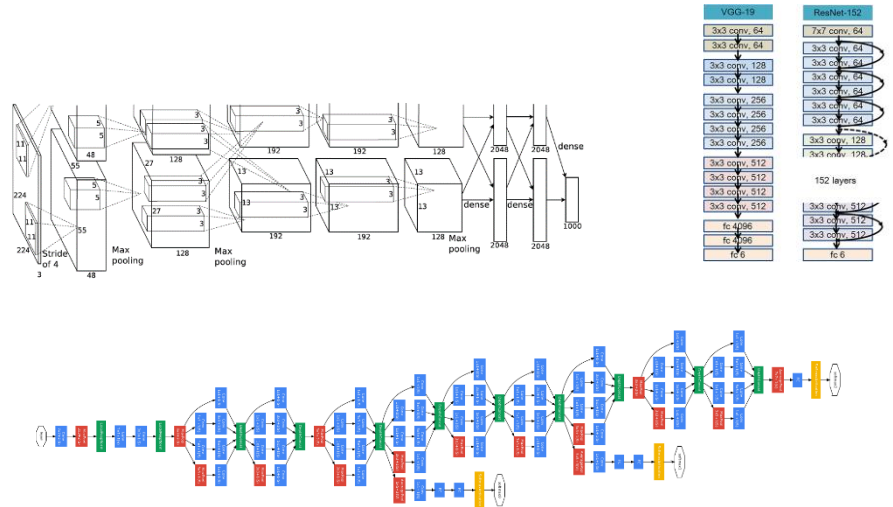


Image Segmentation

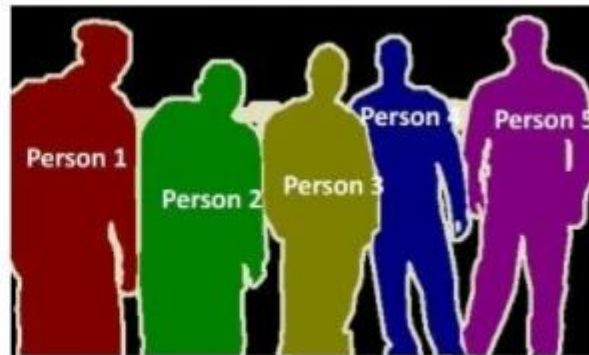
Image Segmentation

➤ There are two main types of image segmentation:

1. Semantic Segmentation
2. Instance Segmentation



Semantic Segmentation



Instance Segmentation

Semantic Segmentation

- The goal of semantic **image segmentation** is to **label each pixel of an image** with a corresponding class of what is being represented.



Person
Bicycle
Background

Semantic Segmentation

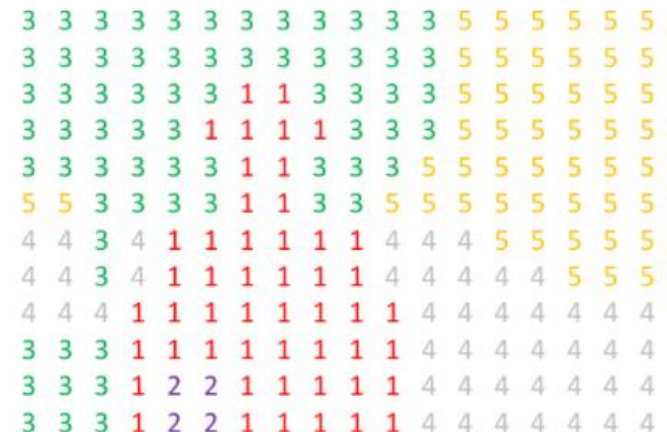
- output a segmentation map where each pixel contains a class label represented as an integer



Input



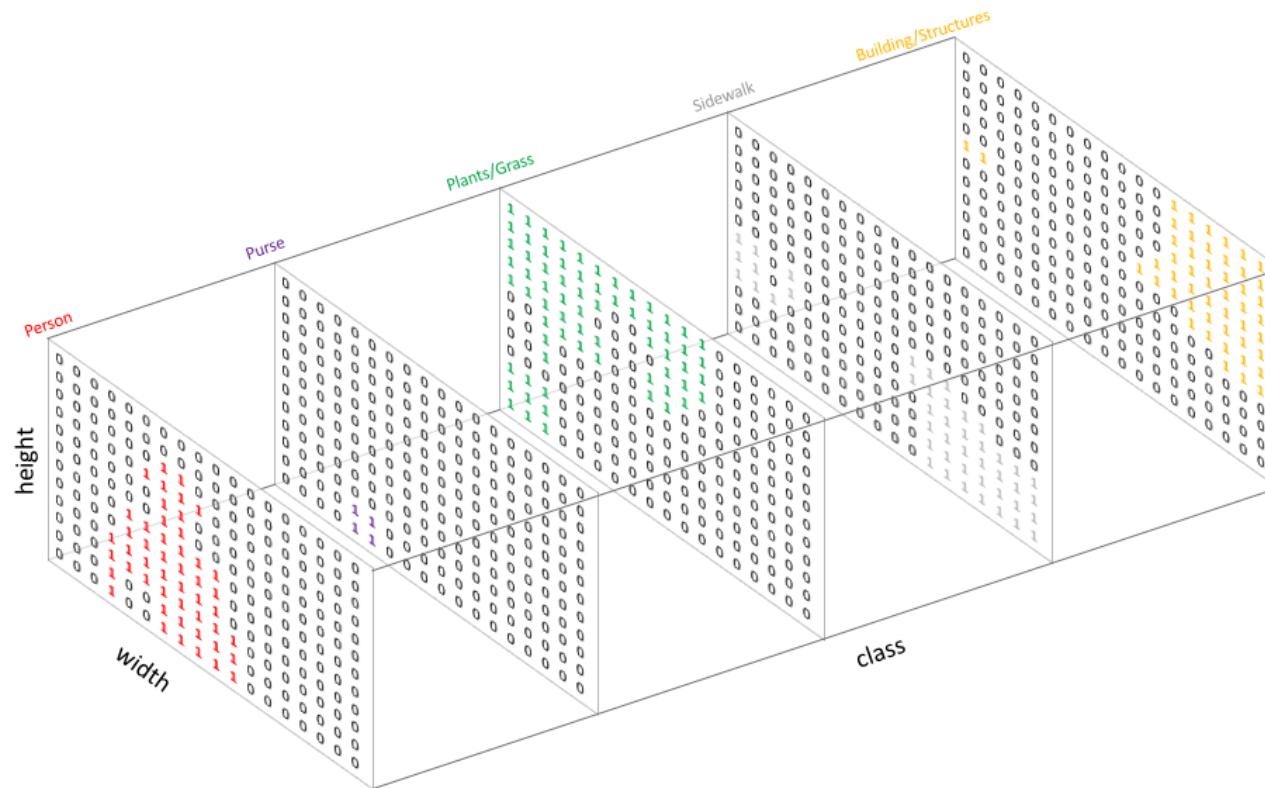
1: Person
2: Purse
3: Plants/Grass
4: Sidewalk
5: Building/Structures



Semantic Labels

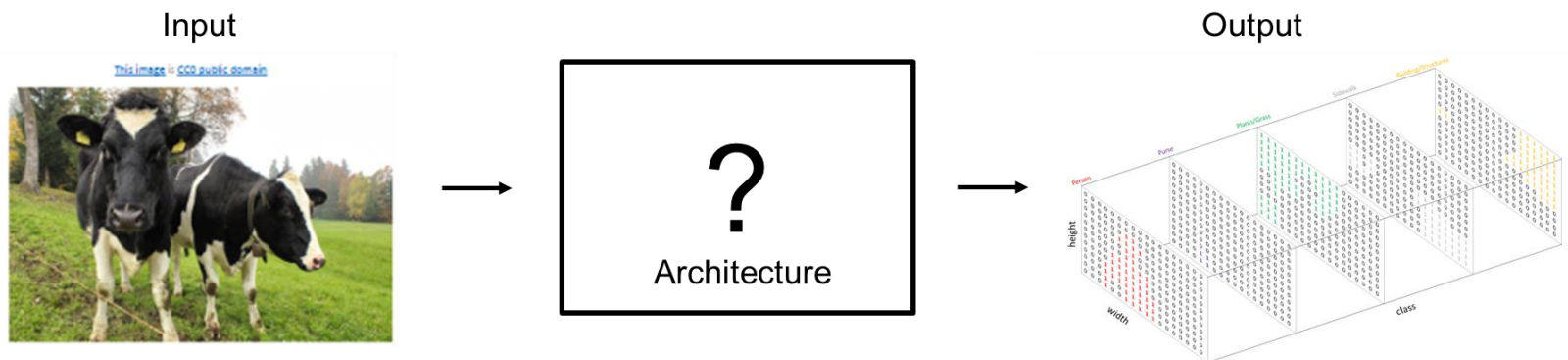
Semantic Segmentation

➤ This is then converted into a one-hot encoding



Selecting an Architecture

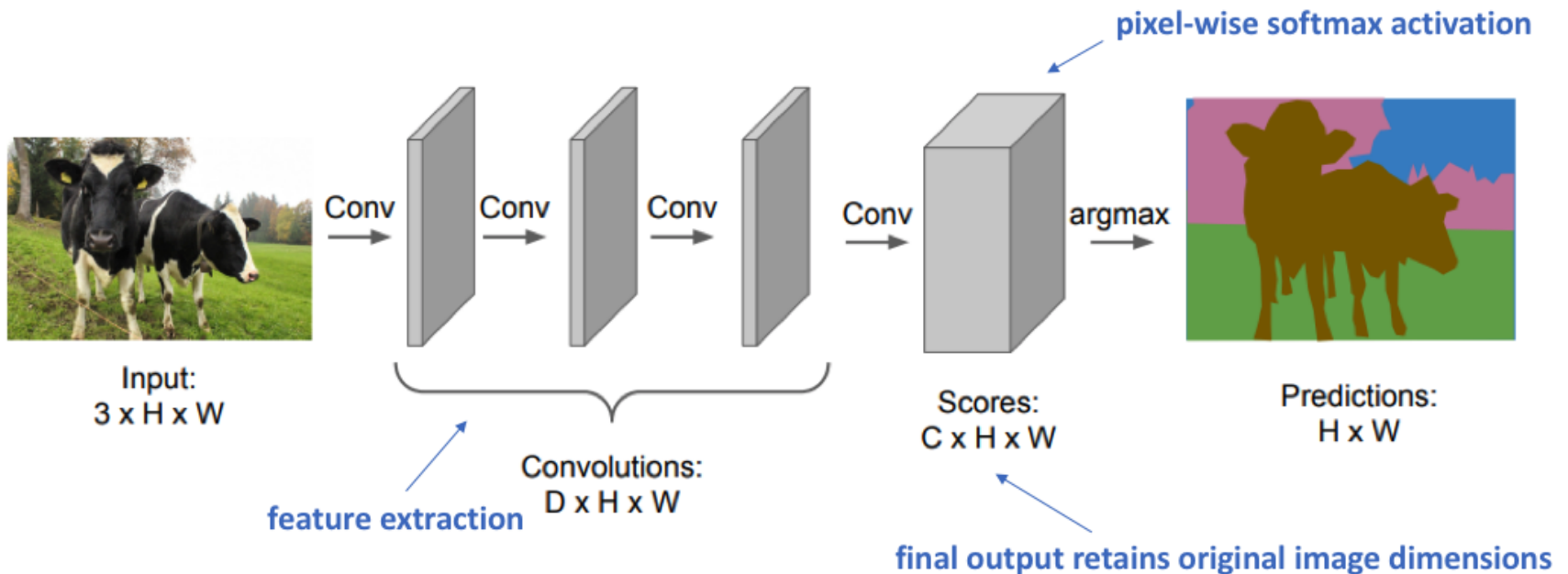
- We know what the output should be... we know what the input should look like...



- Q: What should the architecture look like?

Naïve Architecture

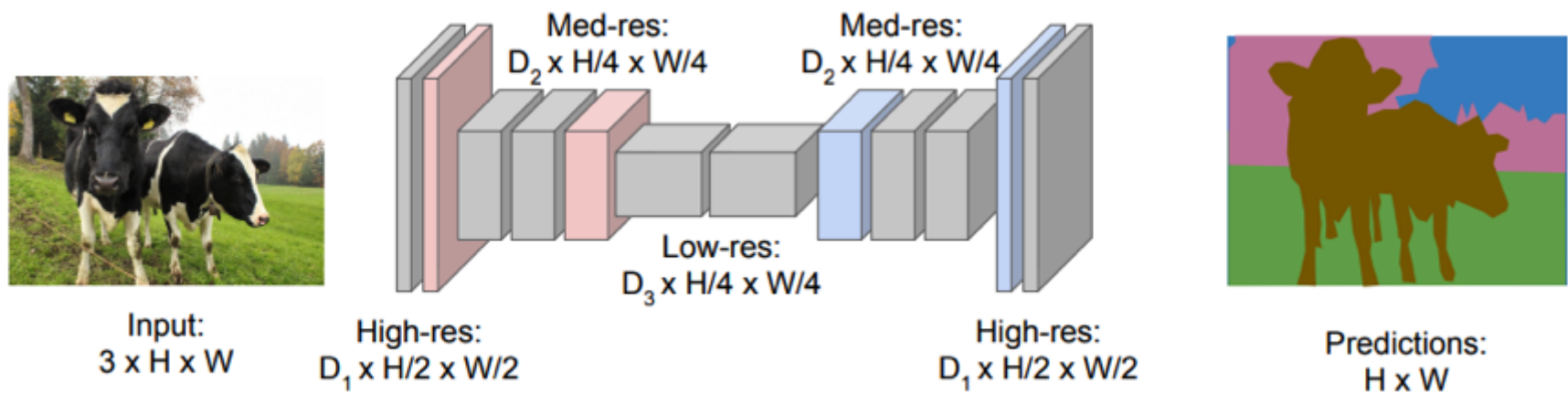
Solution: Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Downside: Preserving image dimensions throughout the network will be computationally expensive.

Better Approach

Solution: Make network deep and work at a lower spatial resolution for many of the layers.

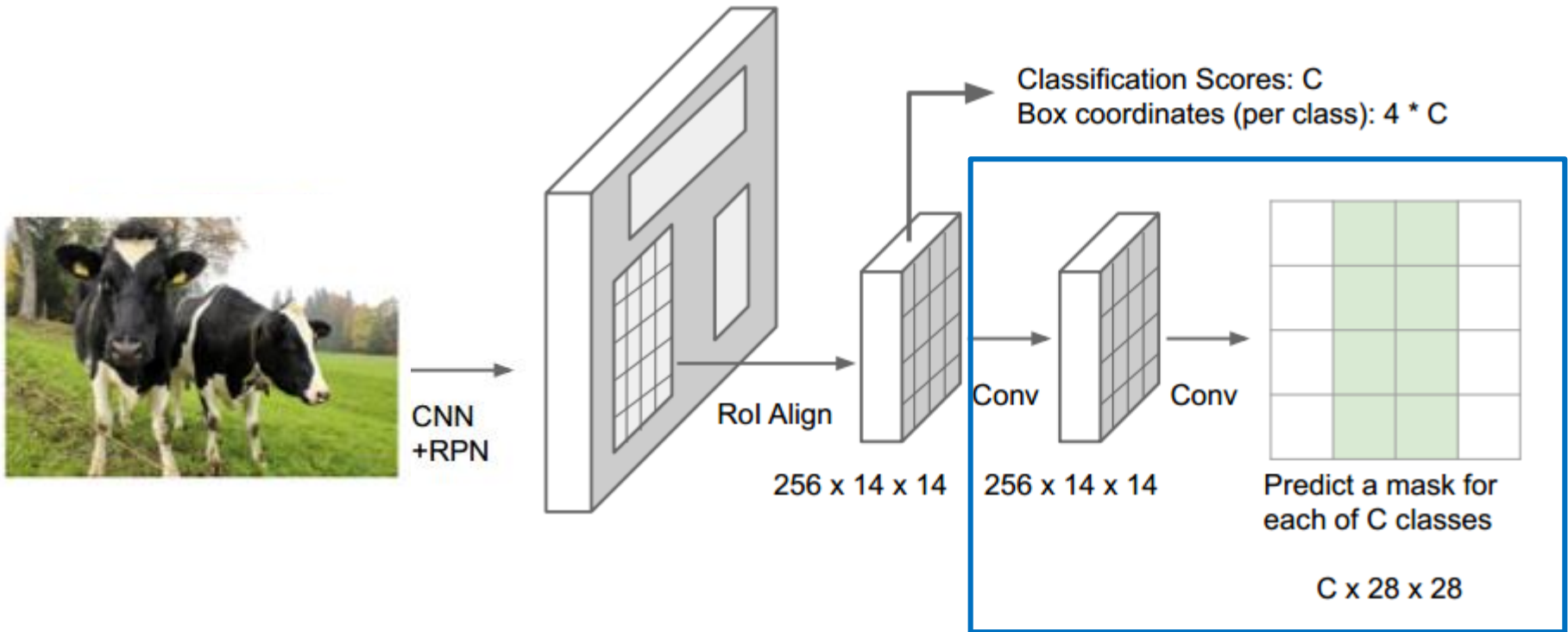


Requirements: Bunch of convolutional layers with **downsampling** and **upsampling** inside the network!

[more on this today ...](#)

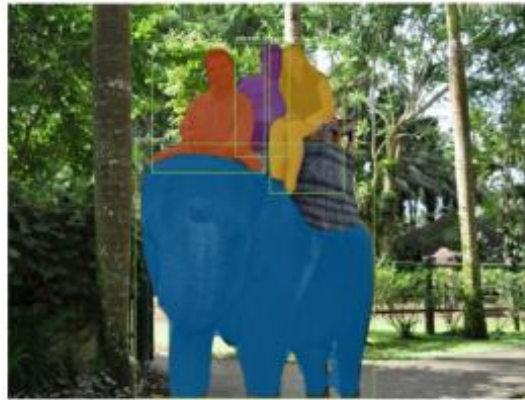
Source: Noh et al, ICCV 2015

Mask R-CNN

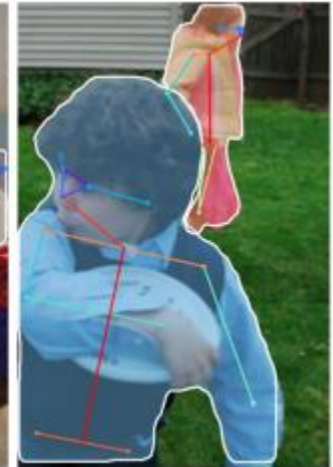


R-CNN architecture with additional branch to predict a binary mask

Mask R-CNN



Can also do pose estimation...



Source: He et al, 2017

Summary: Computer Vision Tasks

Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

Motivation

Q: What is one thing that all the learning models covered up to now have had in common?

Motivation

Challenges with Supervised Learning:

- Requires large amounts of labeled data
- Obtaining labeled data is expensive
 - e.g., medical tests are expensive because they require a specialist to review them.
- Often there is a lot more unlabeled data than labeled.
- Not what we see in biology

Motivation

Q: How do humans learn?

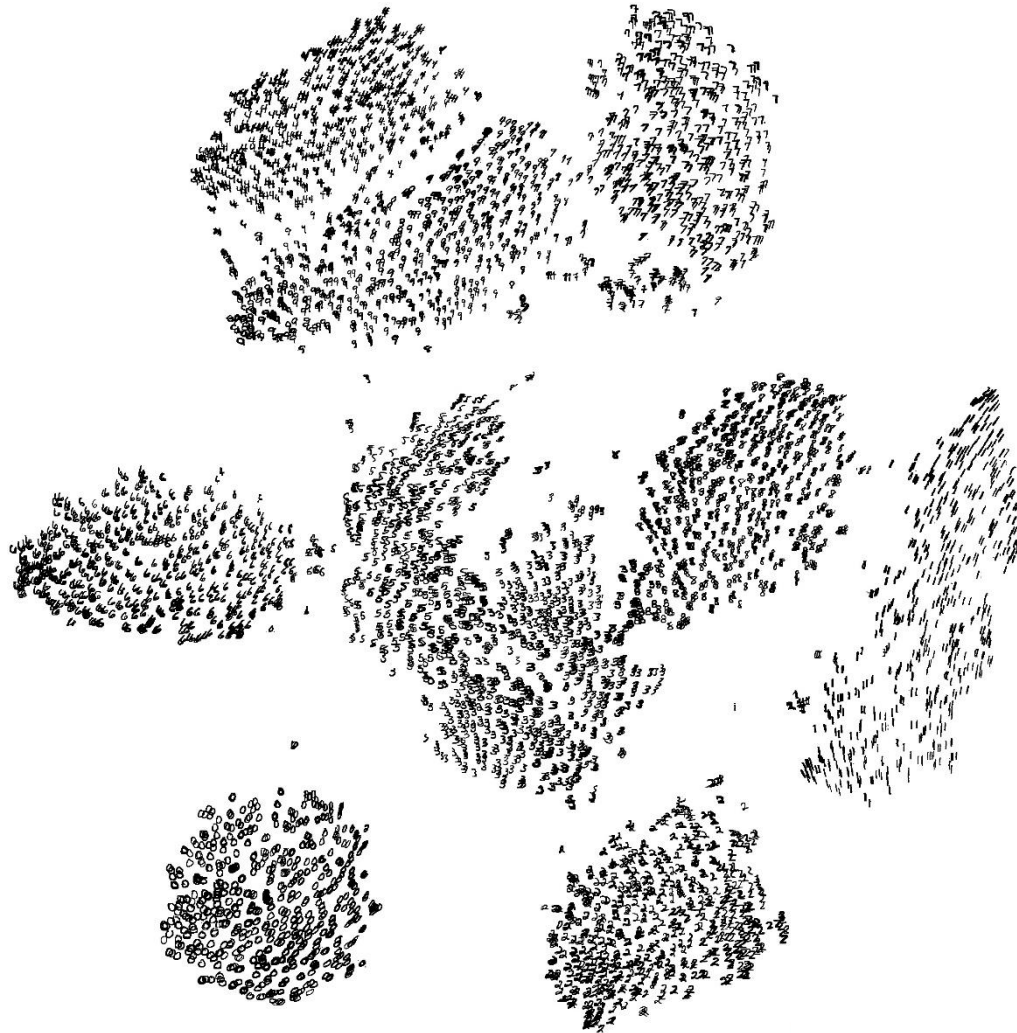
More specifically how do babies learn?

Motivation

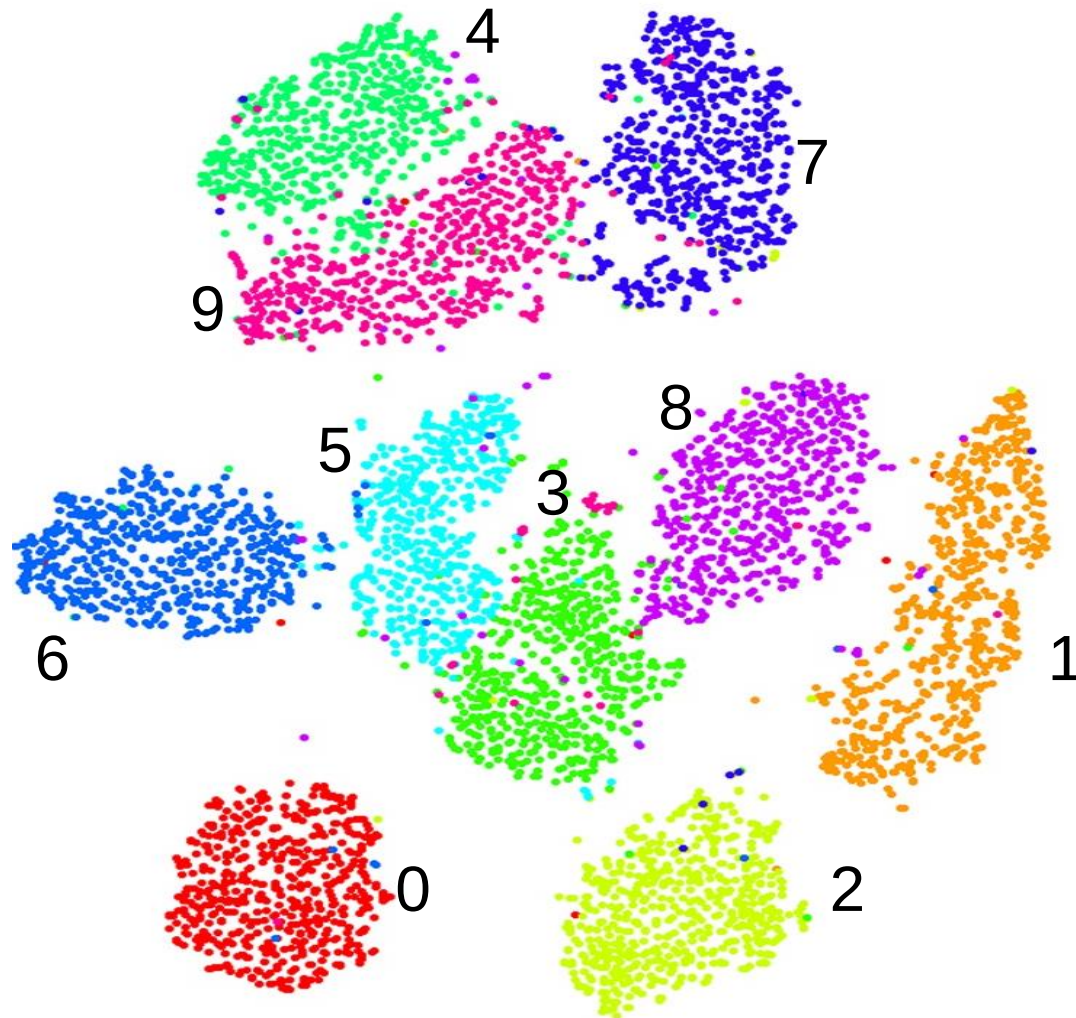
Unsupervised Learning:

- Our brains are constantly observing the world around us for patterns, or some structure to relate objects.
- Patterns or clusters of similar features can tell us a great deal about the data before we even have a label.

Feature Clustering



Feature Clustering



Motivation: Pattern

➤ Which of the following number sequences do you find the easiest to memorize?

- 44, 20, 24, 16, 80
- 55, 89, 144, 233, 377, 610, 987
- 19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 31

Memory and Pattern Matching

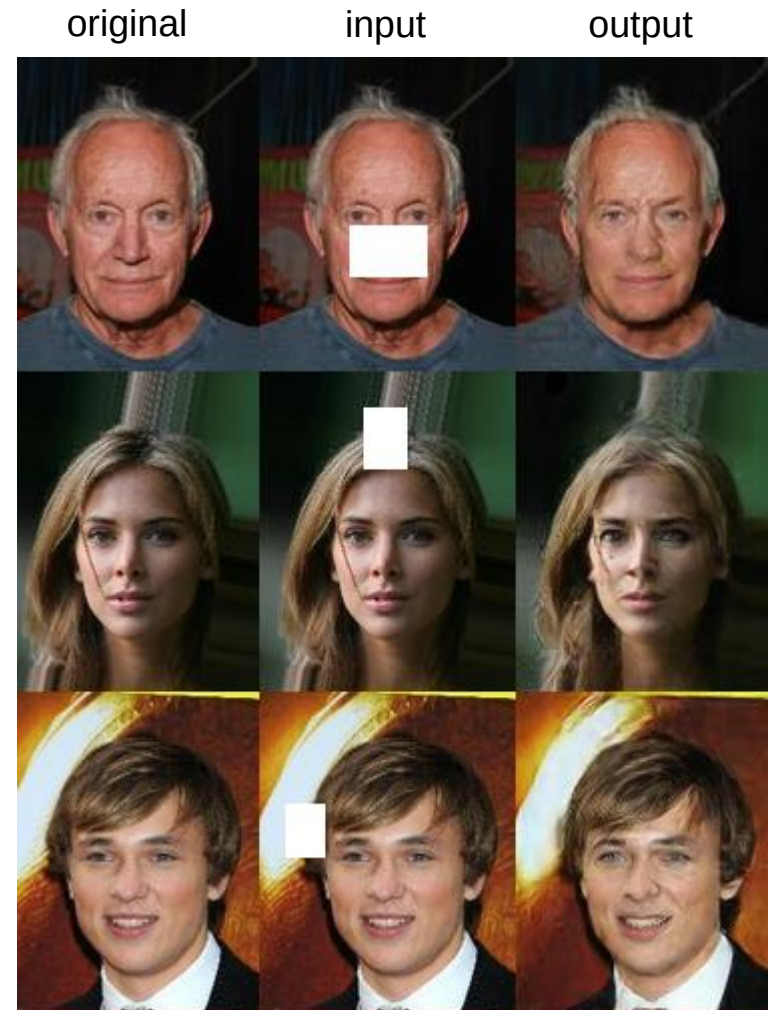
- Professional chess players are able to memorize the positions of all the pieces in a game in 5 seconds.
- Q: Most people would find this impossible, so how are chess players able to do it?

Autoencoders

- Find efficient representations of input data that could be used to reconstruct the original images.
- Composed of **two parts**:
 - **encoder** converts the inputs to an internal representation (dimensionality reduction)
 - **decoder** converts the internal representation to the outputs (generative network)

Applications of Autoencoders

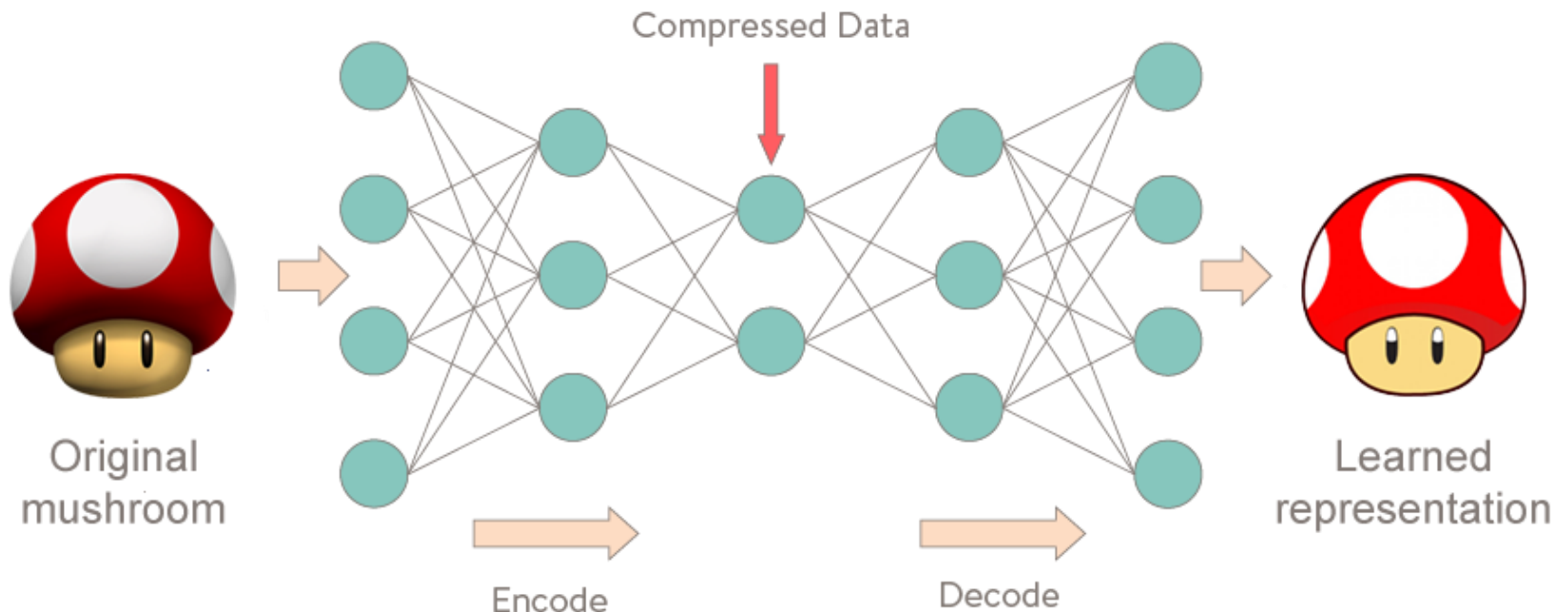
- Feature Extraction
 - unsupervised pretraining
- Dimensionality Reduction
- Image Denoising
- Anomaly detection
 - autoencoders are bad at reconstructing outliers
- Generate new data
 - after some modifications...



Source: [Avery Allen and Wenchem Li](#)

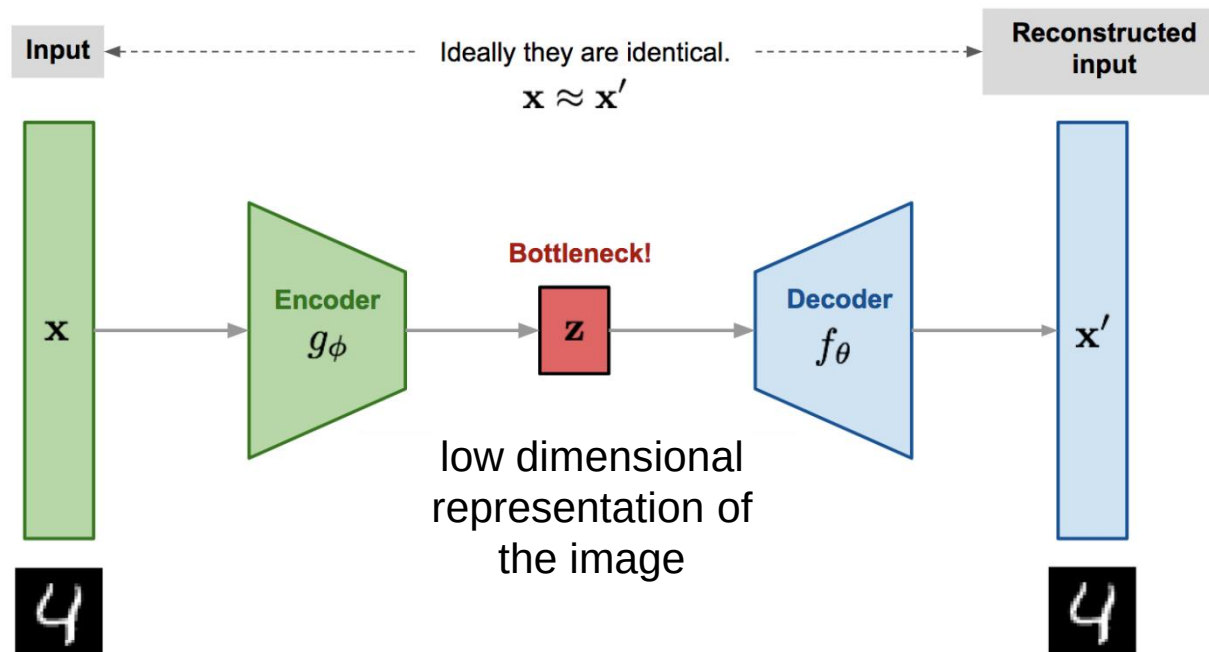
Autoencoder Architecture

- Typically, the number of outputs is the same as the inputs
- Hourglass shape creating a bottleneck layer or lower dimensional representation



Unsupervised Learning

- Autoencoders are not used for supervised learning. The task is **not** to predict something about the data!
- It is forced to learn the most important features in the input data and drop the unimportant ones



How to Train Autoencoders?

➤ **Loss function:**

- Goal: Determine how close was a reconstructed image to the original?
- Solution: Mean square error across all the pixels (input vs output) – aka MSE Loss.

➤ **Optimizer:**

- Just like before!
- Typically use Adam optimizer

➤ **Training loop:**

- Just like before!

PyTorch Implementation

```
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()
        encoding_dim = 32
        # encoder
        self.fc1 = nn.Linear(28 * 28, encoding_dim)
        # decoder
        self.fc2 = nn.Linear(encoding_dim, 28 * 28)

    def forward(self, img):
        flattened = img.view(-1, 28 * 28)
        x = self.fc1(flattened)
        # sigmoid for scaling output from 0 to 1
        x = F.sigmoid(self.fc2(x))
        return x

criterion = nn.MSELoss()
```


Unsupervised Learning

- Q: There is a loss function, and we have “labels” that we’re trying to match. So why is this NOT considered supervised learning?

Unsupervised Learning

➤ A:

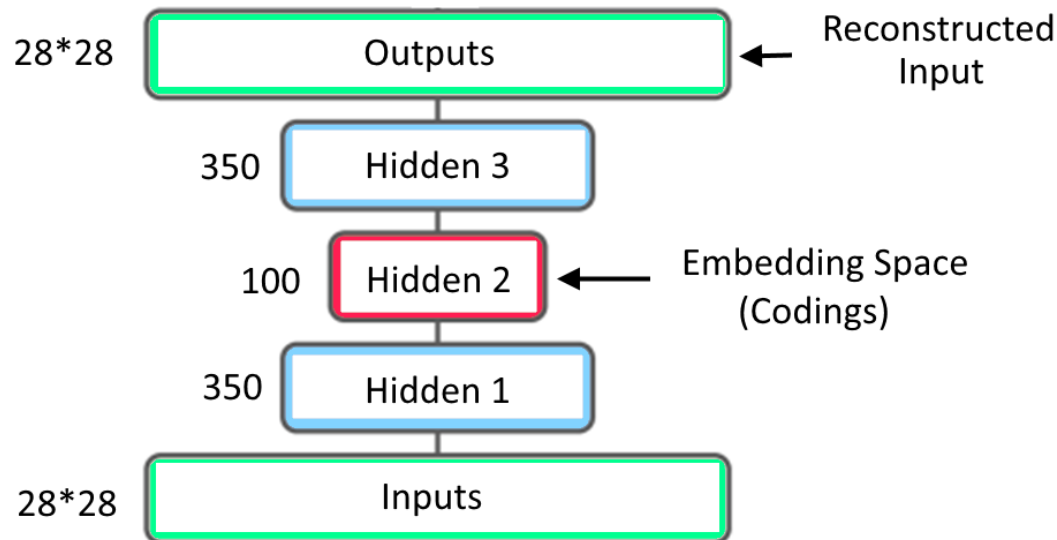
- Yes, there is a loss function
- There is an **auxiliary task** that we know the answer to
- But there is no “label” or “ground truth” with respect to the **actual task** that we want to accomplish.

➤ Referred to also as **Self-Supervised Learning**

Deep Autoencoders

Stacked (Deep) Autoencoders

- Autoencoders can have multiple hidden layers.
- Called stacked autoencoders (or deep autoencoders)
- The architecture of a stacked autoencoder is typically symmetrical with regards to the central coding layer.
- eg.,

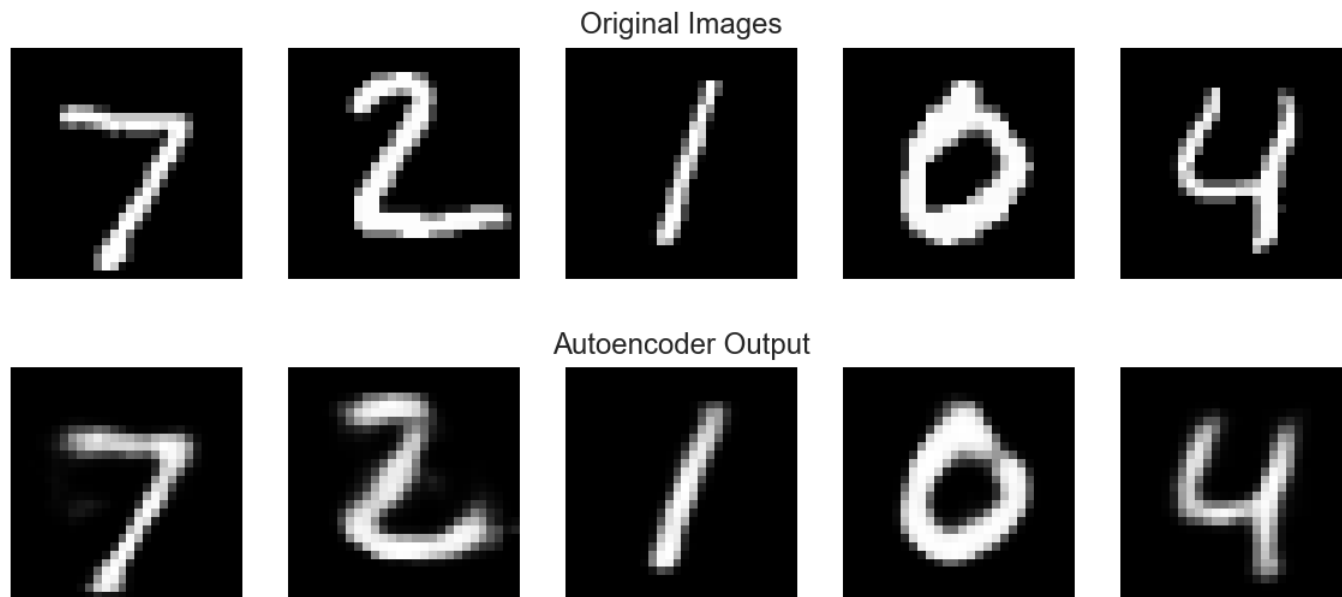


Risk of Overfitting

- You could easily train deeper autoencoders this way...
- Potential for Overfitting!
 - If encoder is too deep, it could map each input to a single arbitrary number
 - Such an encoder would reconstruct the training data perfectly and would not have learned any useful representations
 - Unlikely to generalize to new instances.

Visualizing Reconstructions

- One way to ensure that an autoencoder is properly trained is to compare the inputs and the outputs.
- Let's plot random digits and their reconstructions:

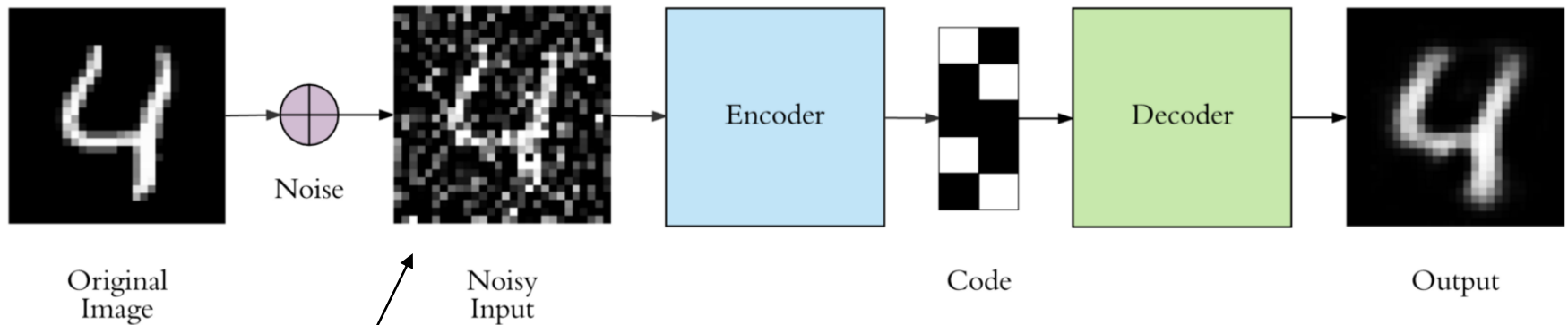


Perfect reconstruction does not rule out overfitting

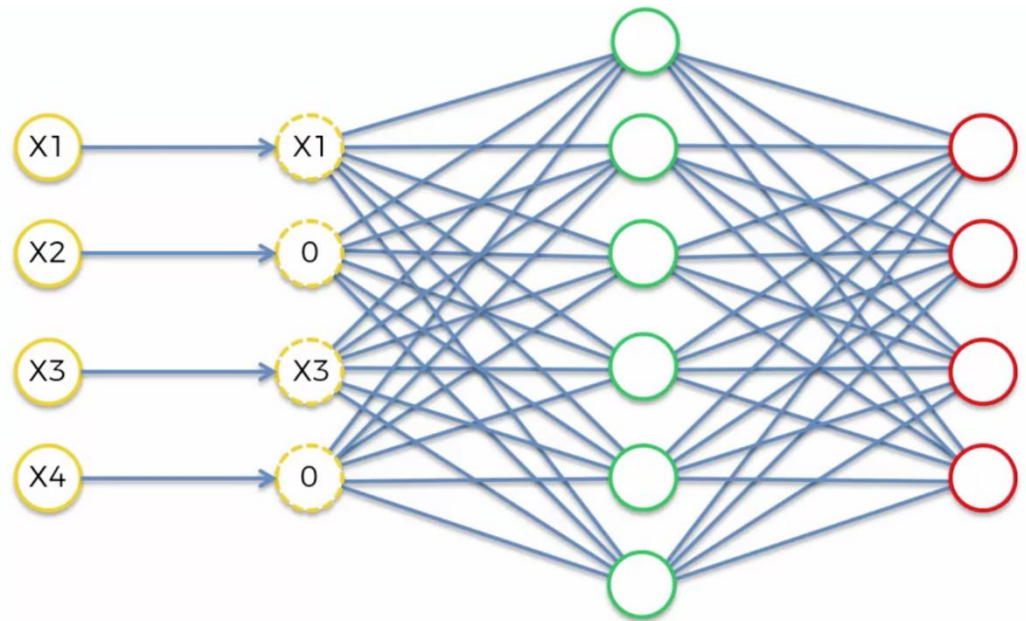
Denoising Autoencoders

- Noise can be added to the input images of the autoencoder to encourage it to learn more useful features
 - Trained to recover the original, noise-free inputs.
- Prevents the autoencoder from trivially copying its inputs to its outputs (i.e., autoencoder must find patterns in the data).

Denoising Autoencoders



Add (1) Gaussian noise or
(2) zero out randomly selected inputs using dropout



PyTorch Implementation

```
# how much noise to add to images
```

```
nf = 0.4
```

```
# add random noise to the input images
```

```
noisy_img = img + nf * torch.randn(*img.shape)
```

```
# Clip the images to be between 0 and 1
```

```
noisy_img = np.clip(noisy_img, 0., 1.)
```

labels/targets
are the clean
images!

```
# compute predicted outputs using noisy_img
```

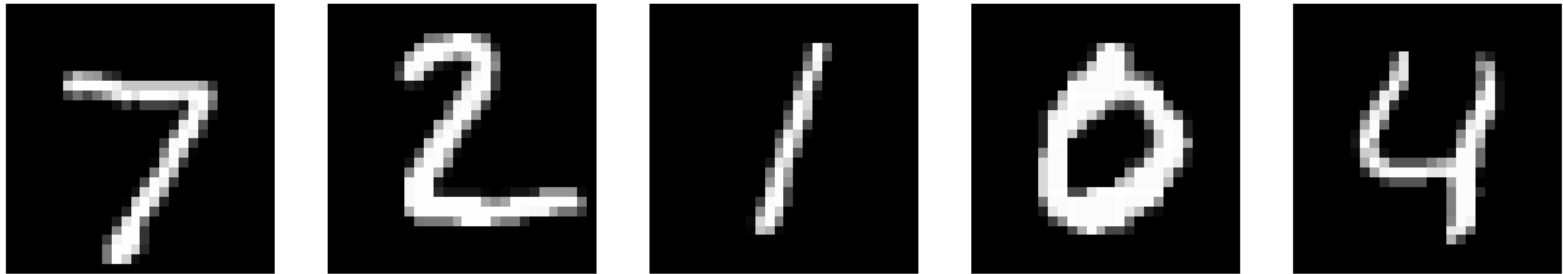
```
outputs = model(noisy_img)
```

```
# the target is the original img
```

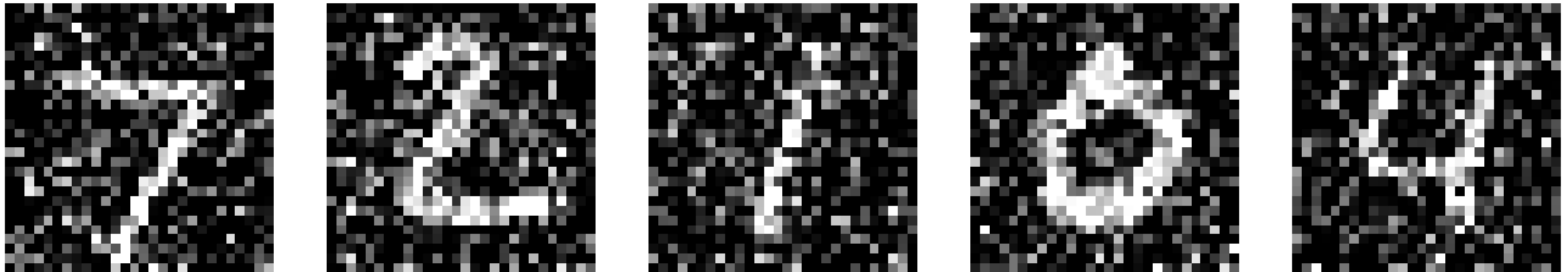
```
loss = criterion(outputs, img)
```

Denoising Autoencoder Results

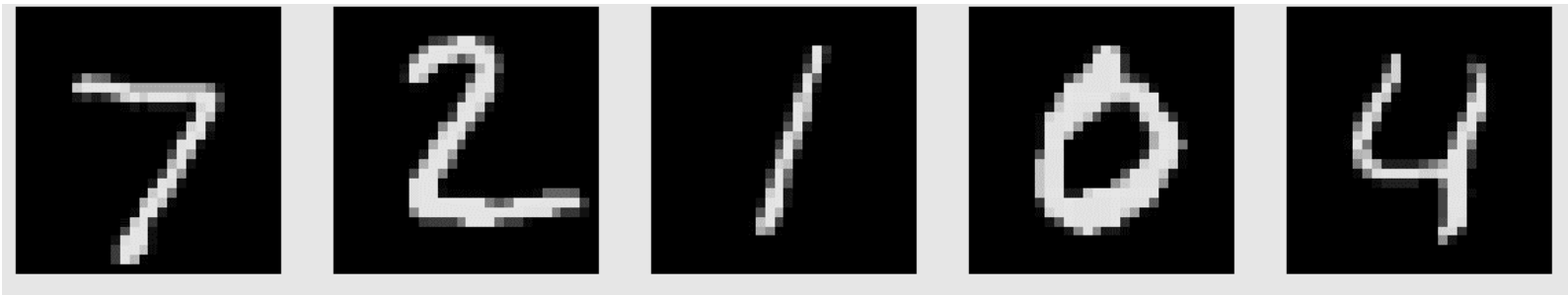
Original image



Noisy image



Reconstructed image



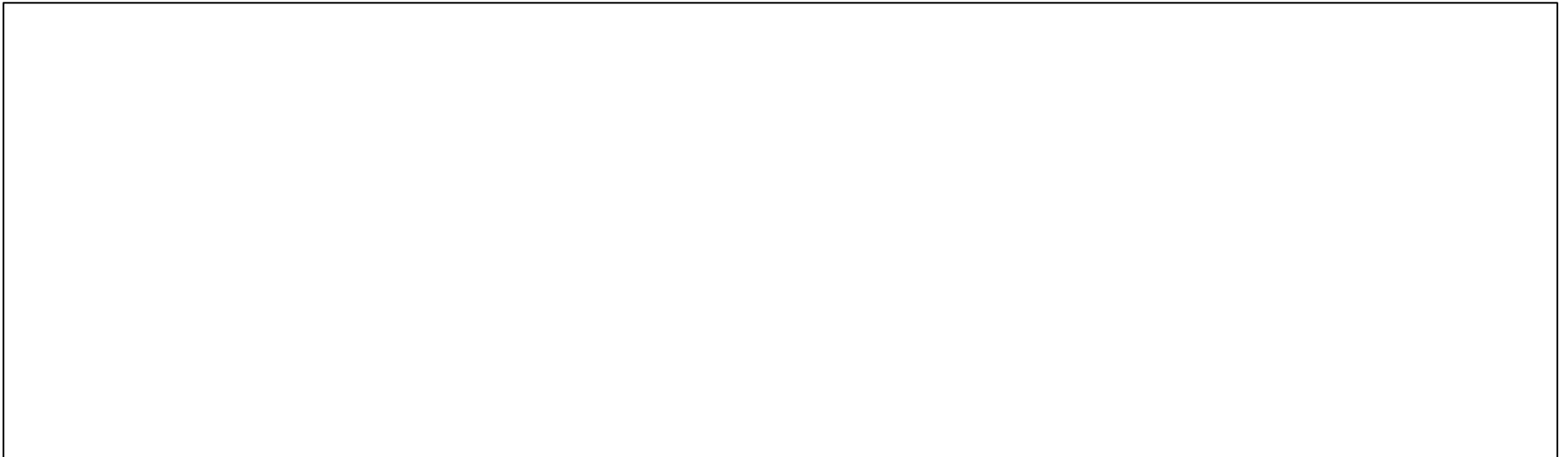
Generating New Images

Generate New Images

- Since we are drastically reducing the dimensionality of the image, there has to be some kind of structure in the codings (i.e. **embedding space**).
- That is, the network should be able to **"save" space by mapping similar images to similar embeddings**.
- Let's see how we can exploit this to allow us to generate new types of images.

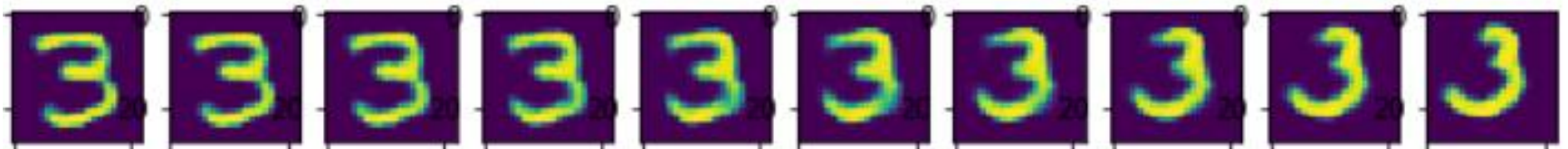
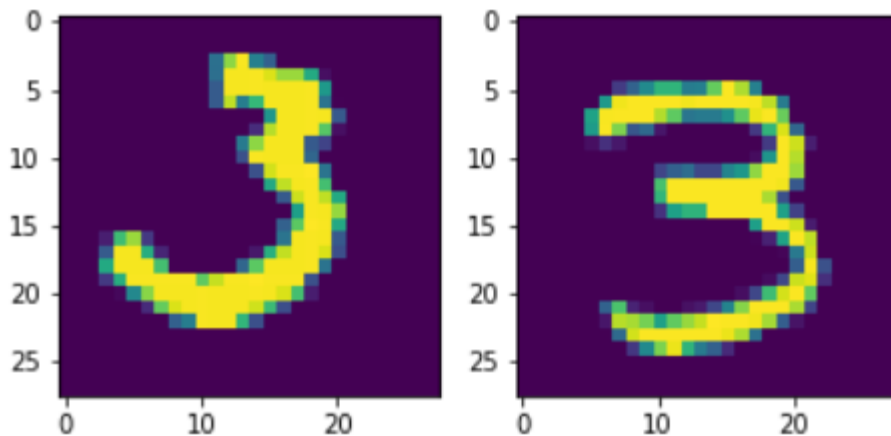
New Images with Interpolation

- First compute **low-dimensional embeddings** of two images
- Then interpolate between the two embeddings and decode those as well!



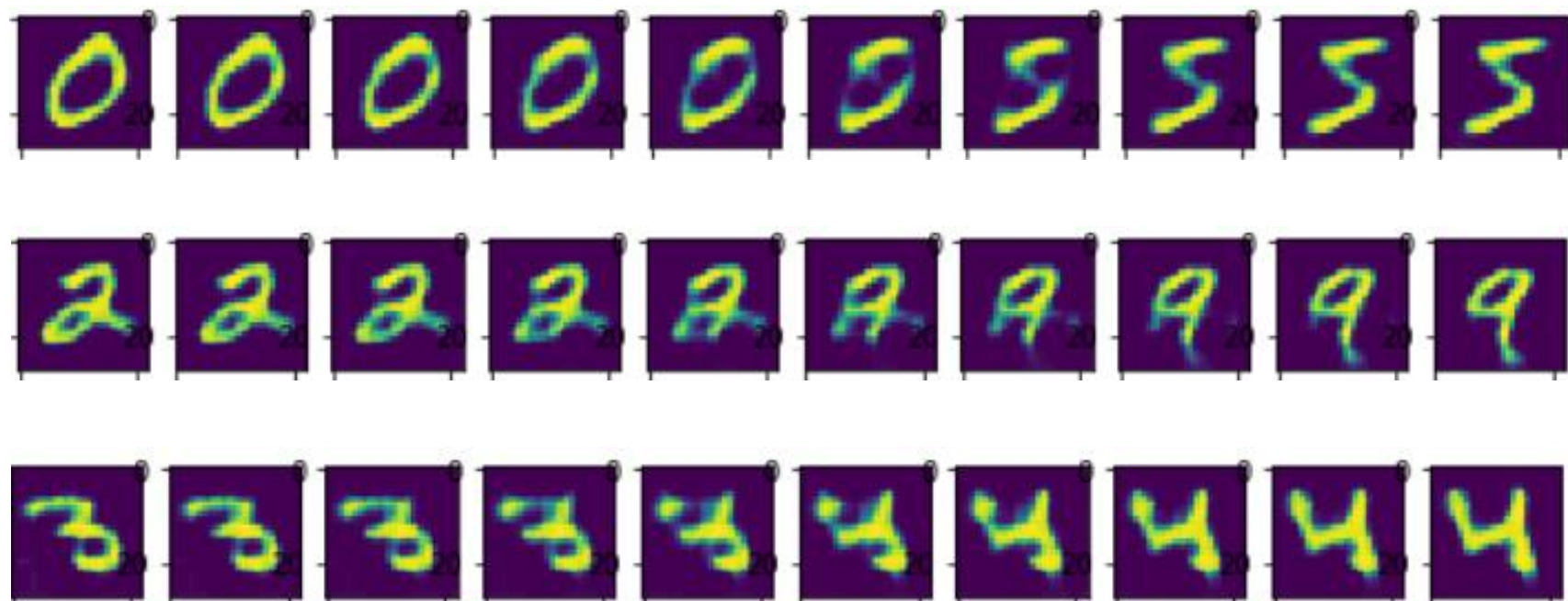
Plotting Interpolated Codings

- Interpolated codings result in new images that are somewhere in between the two starting images.



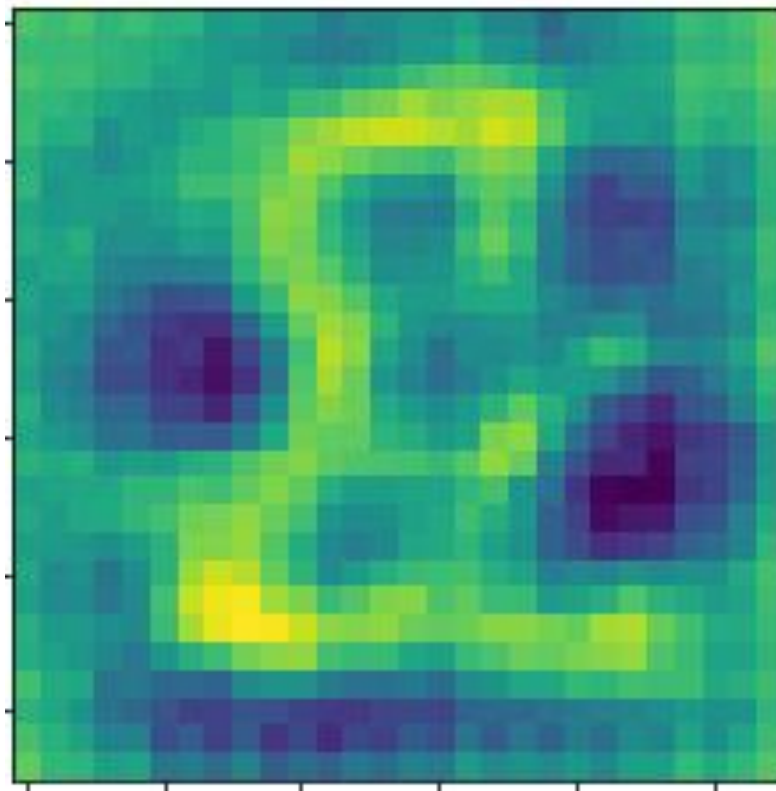
Plotting Interpolated Codings Con't

- We can do this for other image combinations



Plotting Interpolated Codings Con't

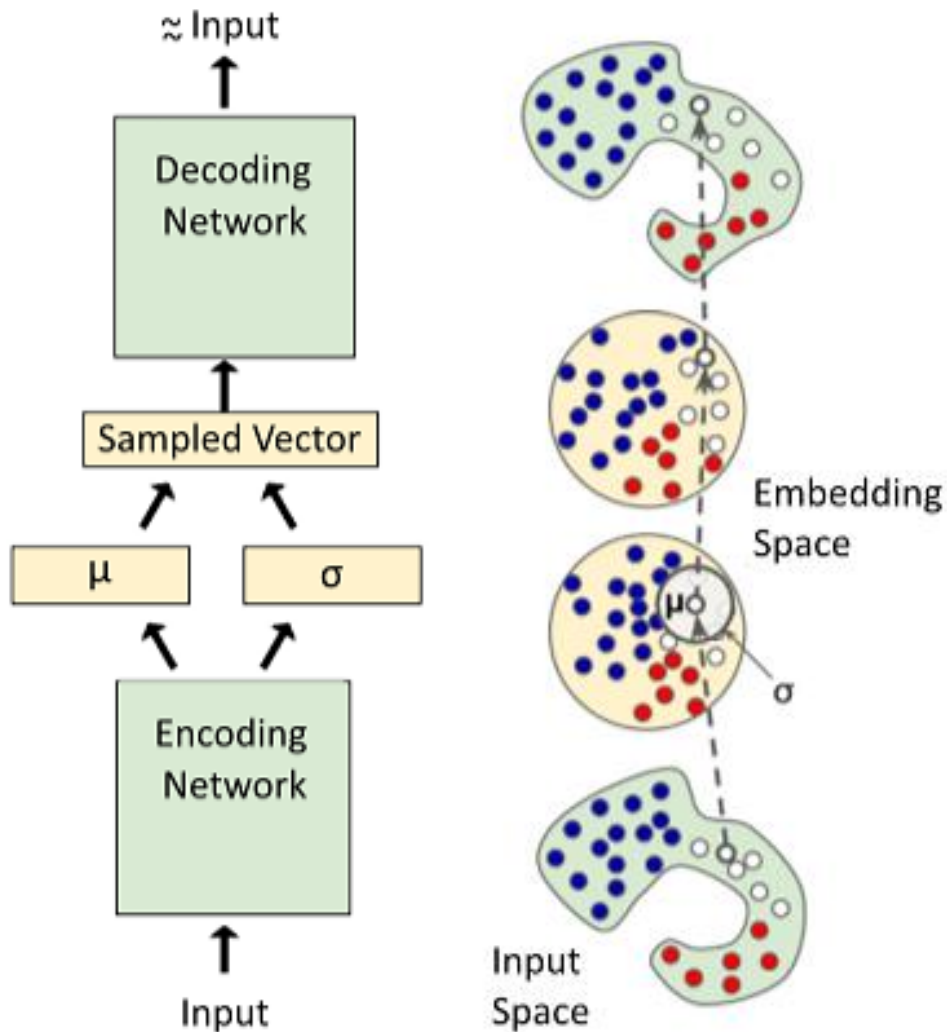
➤ Q: What if we randomly select a coding?



Variational Autoencoders

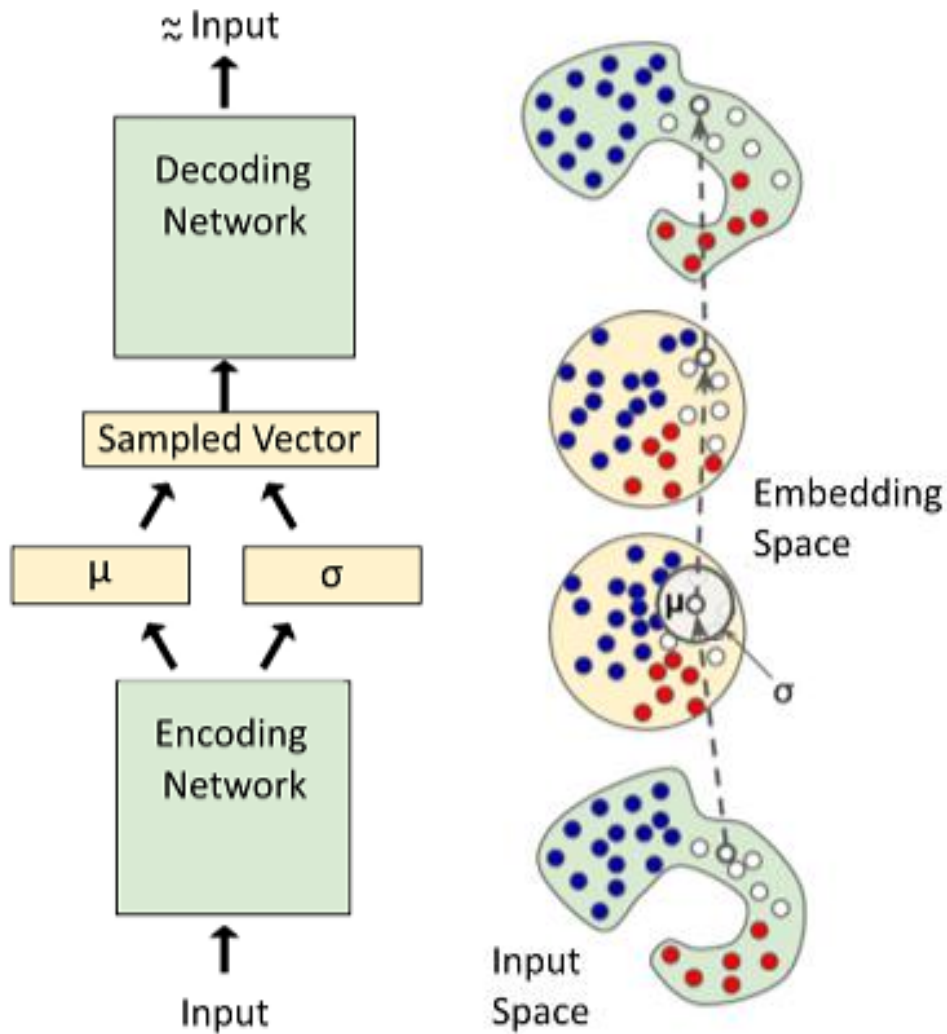
- They are quite different from all the autoencoders we have discussed so far, in particular:
 - ***probabilistic autoencoders***, meaning that their outputs are partly determined by chance even after training
 - ***generative autoencoders***, in that they can generate new instances that look like they were sampled from the training set.

Variational Autoencoders



- Encoder produces a mean coding μ and a standard deviation σ .
- Coding is then sampled from a Gaussian distribution with mean μ and standard deviation σ to obtain latent vector (sampled vector).
- Decoder generates new sample from latent vector.

Variational Autoencoders



➤ Loss Function consisting of two parts:

1. Reconstructed Loss
2. KL Divergence

$$\|x - \tilde{x}\|^2 + KL[N(\mu, \sigma), N(0, I)]$$

Generating Data: MNIST

- Example: generate images that look like handwritten digits by training a variational autoencoder.



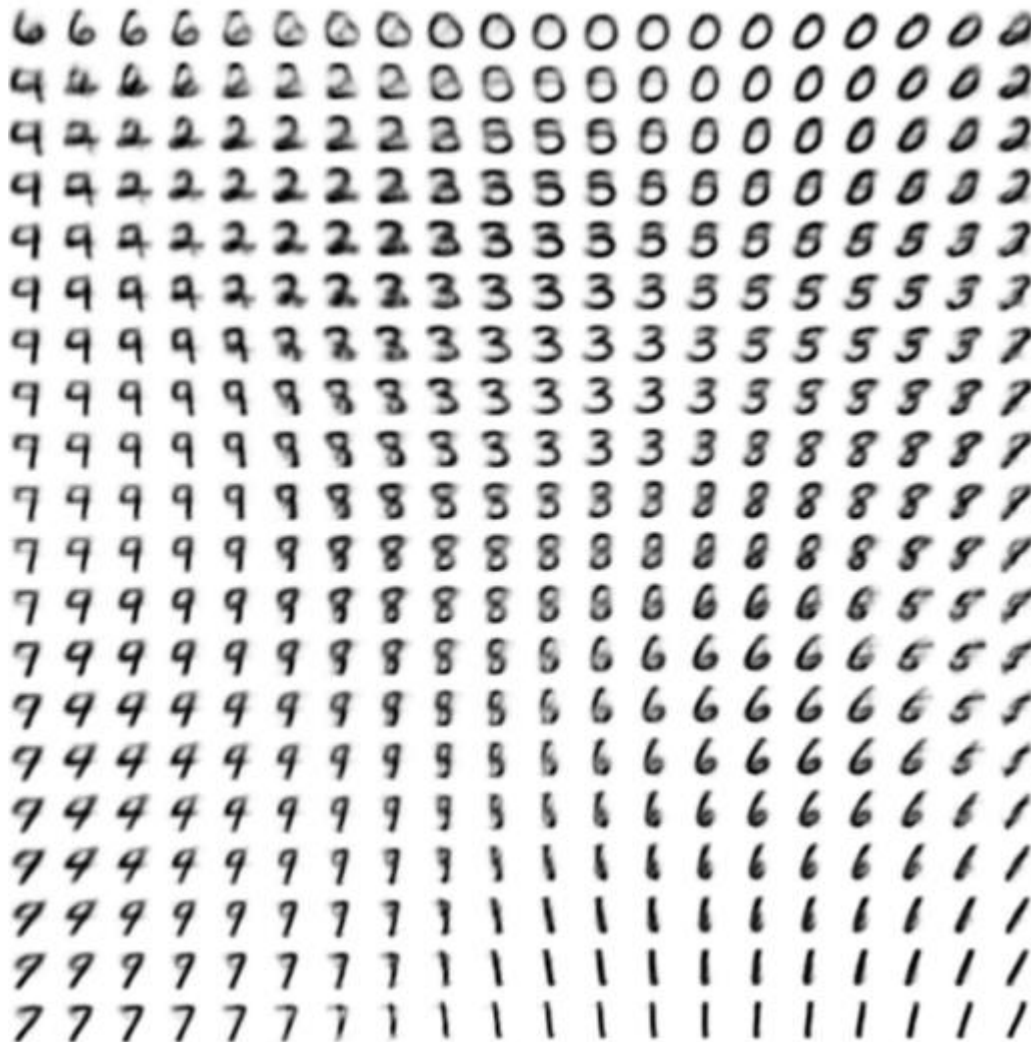
Generating Data: Faces

- Example: generate images that look like real faces.

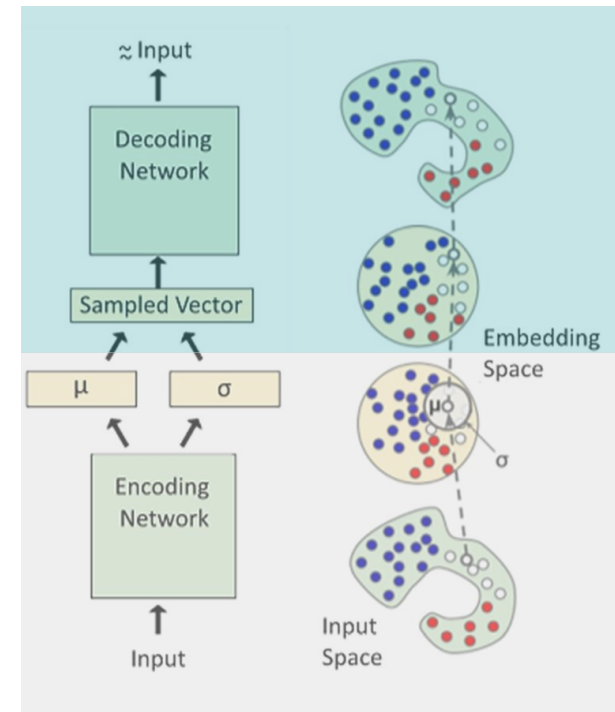


Source: Anders Larsen et al. 2017

Embedding Space: MNIST



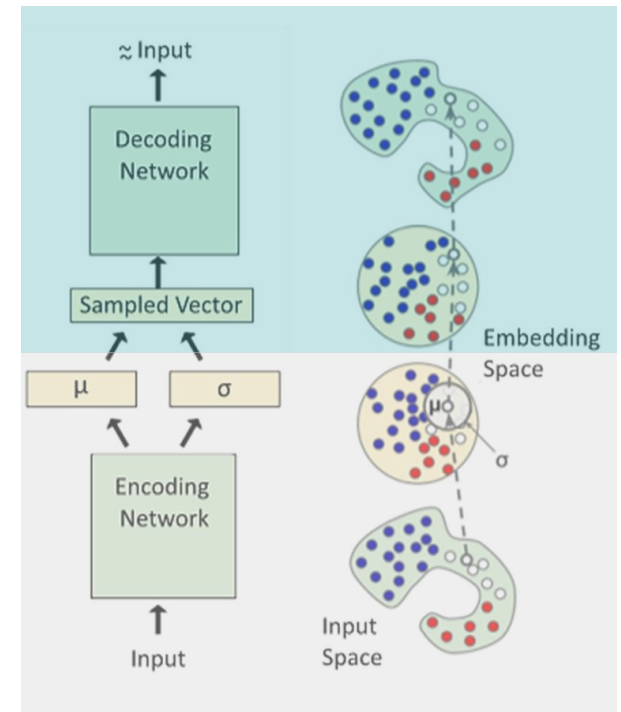
➤ Traversing the embedding space



Embedding Space: Faces



- Traversing the embedding space

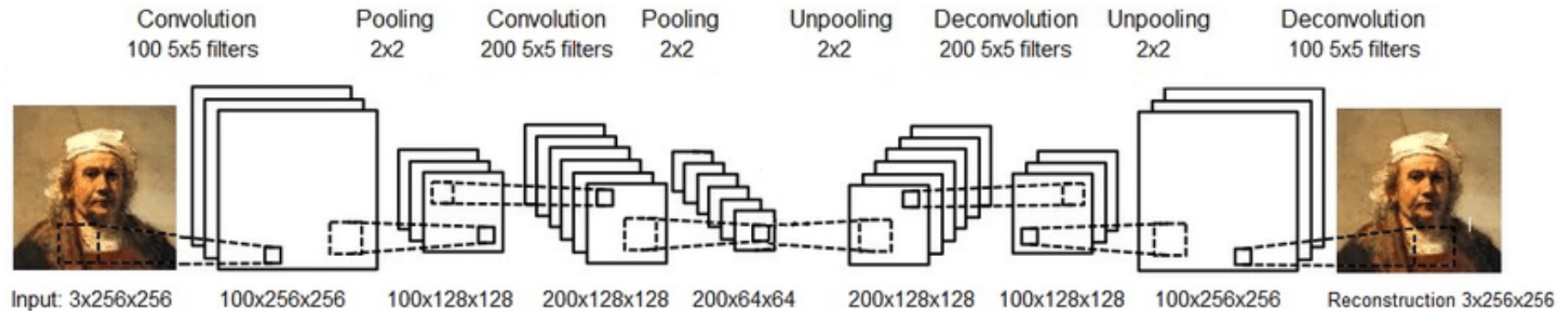


Convolutional Autoencoder

Convolutional Autoencoder

- The autoencoders mentioned up until now did not take advantage of spatial information
- Adding convolutional layers can add improvements when trying to generate image data

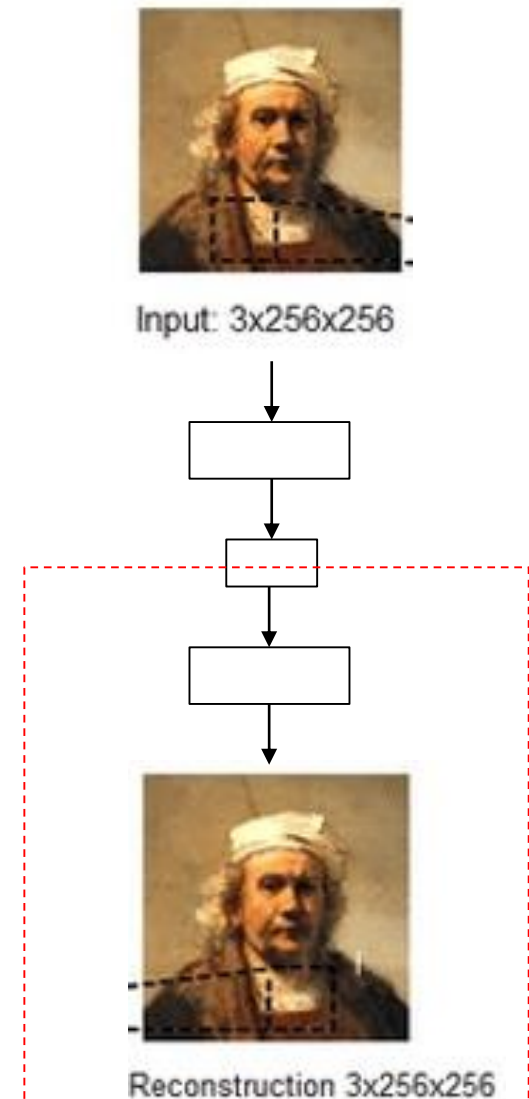
Convolutional Autoencoder



- In a convolutional autoencoder, the encoder works with convolution layers (pooling layers optional).
- The decoder tries to mirror the encoder but instead of "making everything smaller" it has the goal of "making everything bigger" to match the original size of the image.

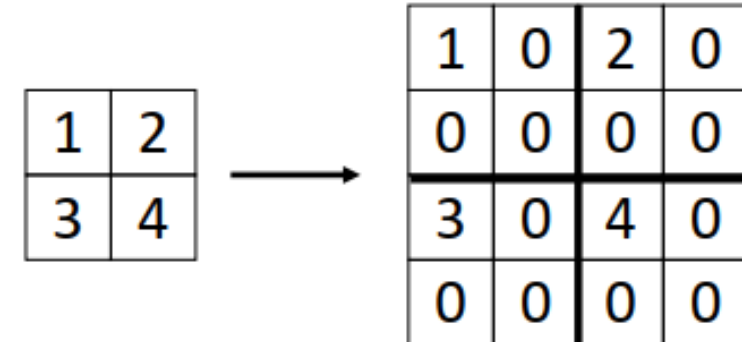
What we need:

- We need to be able to up-sample features (low to high resolution)
 - Opposite of max-pooling OR
 - Opposite of a convolution
- We need a special convolution that results in a larger dimensional output than the input.



Upsampling - Max Unpooling

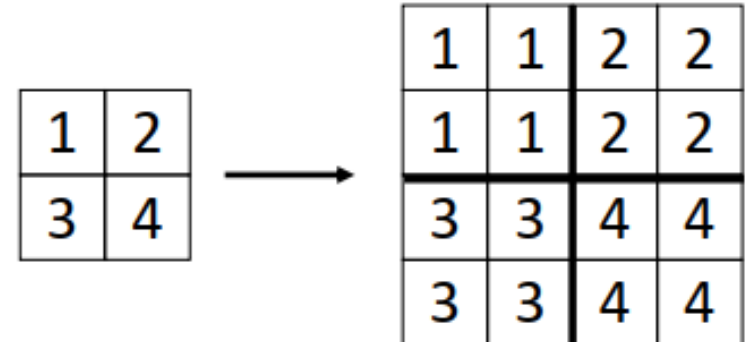
Bed of Nails



Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

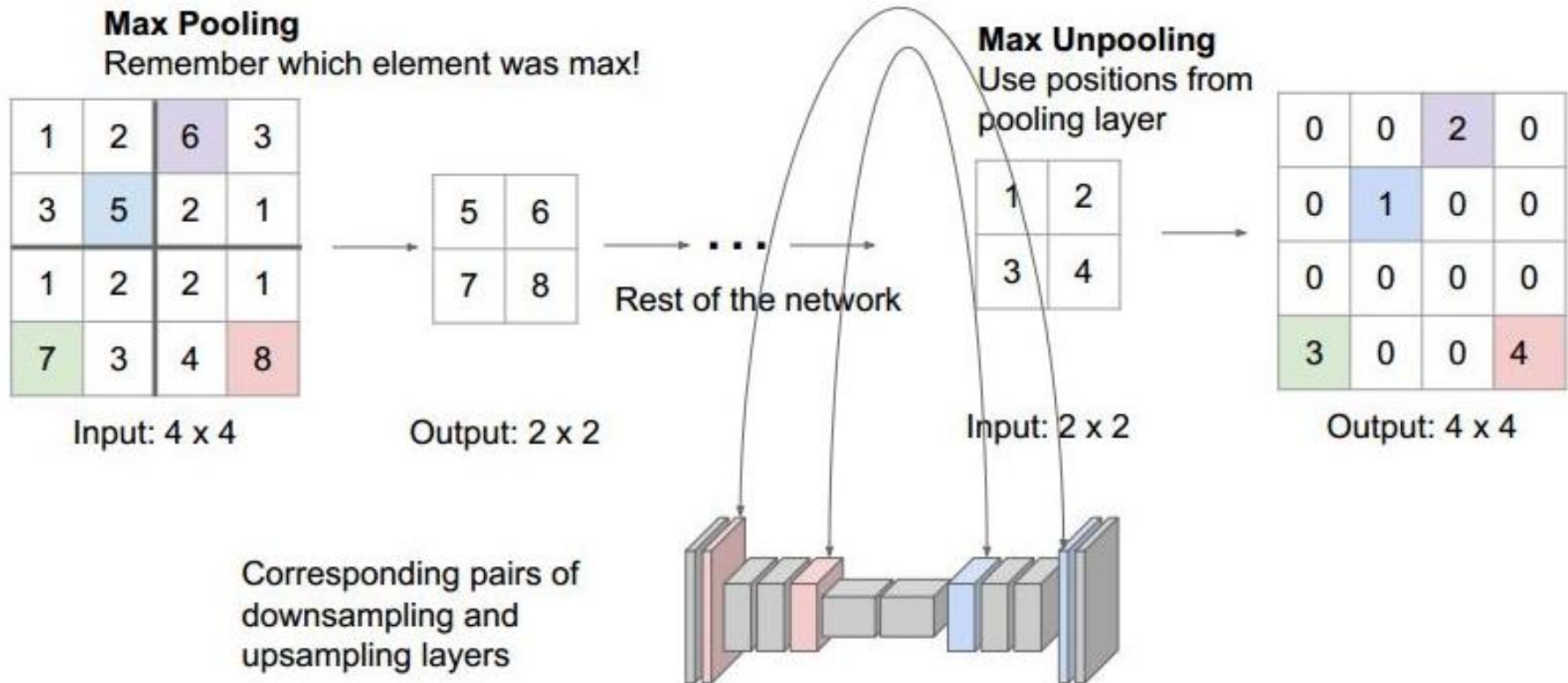
Nearest Neighbor



Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

Upsampling - Max Unpooling

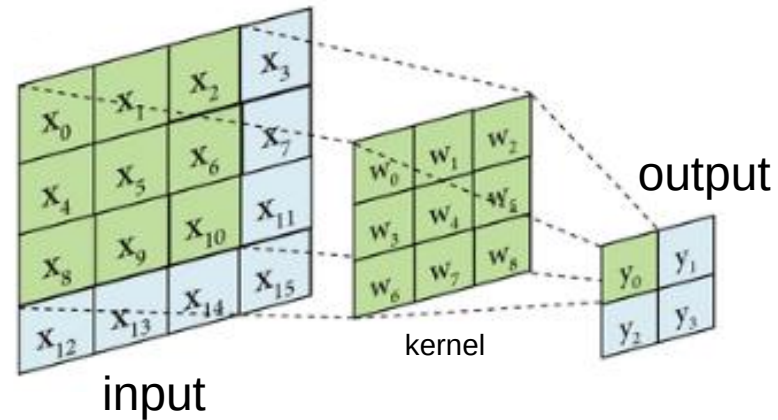


Transposed Convolution

- The opposite of the convolution is the **transposed convolution**.
- Mathematically speaking this is different from an inverse convolution.
- They work with filters, kernels, strides just as the convolution layers, but instead of mapping from e.g., 3x3 input pixels to 1 output, **they can map from 1 input pixel to 3x3 pixels**.

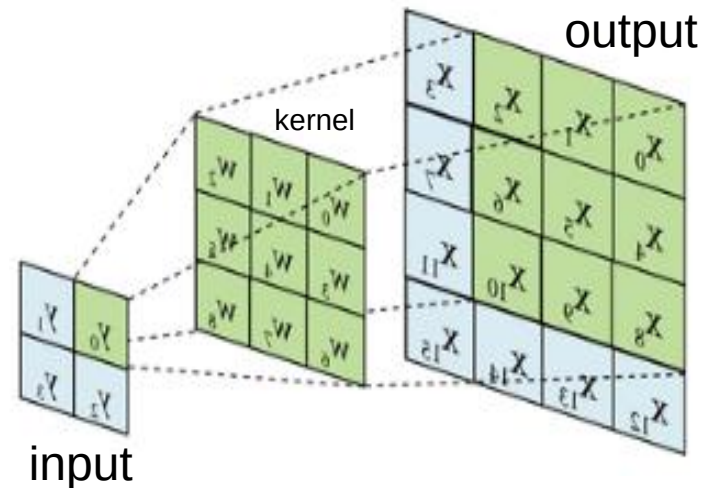
Recap: Convolutional Layers

- take your input image (or feature map)
- calculate the element-wise product with kernel and overlapping “pixels”, then add it all up
- result is a scalar and forms part of the output, then you shift to the next “pixel”.



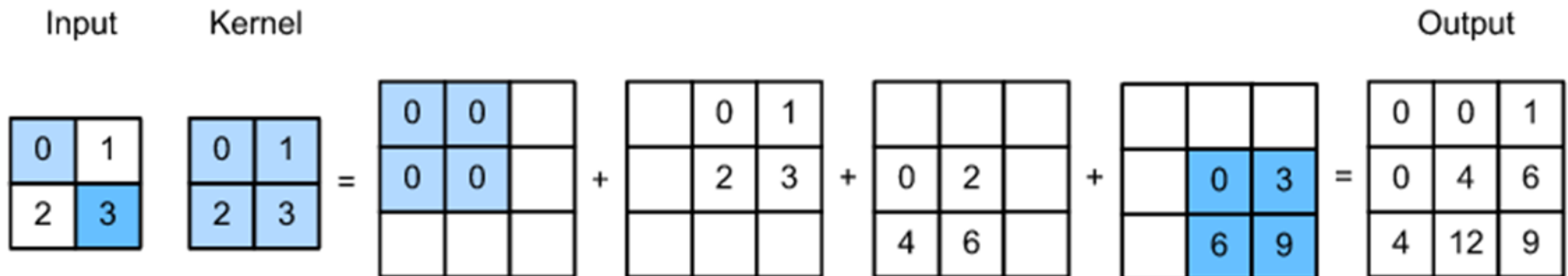
Transpose Convolution

- to do the reverse (i.e. transposed convolution) you take each “pixel” of your input image,
- multiply each value of your kernel with the input pixel to get a weighted kernel
- then you insert this in your output to create an image.
- where the outputs overlap you sum them.



Transpose Convolution Examples

➤ A simple example using a 2x2 kernel is shown



Strides and Padding

- Transpose convolutions also have strides and padding as adjustable settings
- They also have a unique setting referred to as output padding

Transpose Convolution Padding

- The **effect of padding is the opposite** of what happens with the convolution layers
- Instead of adding boundary of 0's to the input, we first compute the output as normal and then remove rows and columns around the perimeter
- Here is a simple example with a 2x2 kernel and a padding of 1:

Input Kernel Output

0	1
2	3

0	1
2	3

 $=$

0	0	
0	0	

 $+$

	0	1
	2	3

 $+$

0	2	
4	6	

 $+$

	0	3
	6	9

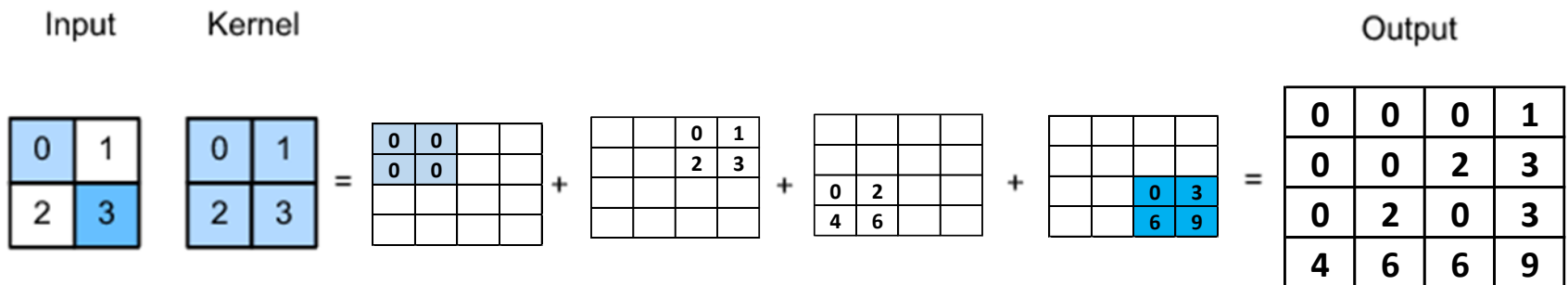
 $=$

4

The diagram illustrates the calculation of a single output element (4) in a 1D transpose convolution. It shows the input [0, 1; 2, 3], a 2x2 kernel [0, 1; 2, 3], and four shifted products: [0, 0; 0, 0], [0, 1; 2, 3], [0, 2; 4, 6], and [0, 3; 6, 9]. The final output is a 1x1 box containing 4.

Transpose Convolution Strides

- The **effect of strides is also the opposite** from what happens with the convolution layers
- Increasing the stride results in an increase in the upsampling effect.
- Here is a simple example with a 2x2 kernel and a stride of 2:



PyTorch Example 1

- A convolution transpose layer with the exact same specifications as the convolution layer would have the “reverse” effect on the shape.

```
conv = nn.Conv2d(in_channels=8,  
                 out_channels=8,  
                 kernel_size=5)
```

```
x = torch.randn(2, 8, 64, 64)
```

```
y = conv(x)
```

```
y.shape
```

```
torch.Size([2, 8, 60, 60])
```

```
convt =  
nn.ConvTranspose2d(in_channels=8,  
                  out_channels=8,  
                  kernel_size=5)
```

```
# should be same as x.shape
```

```
convt(y).shape
```

```
torch.Size([2, 8, 64, 64])
```

PyTorch Example 2

- We also have the option of including convolution transpose padding:

```
convt = nn.ConvTranspose2d(in_channels=16,  
                           out_channels=8,  
                           kernel_size=5,  
                           padding=2)  
  
x = torch.randn(32, 16, 64, 64)  
y = convt(x)  
y.shape  
  
torch.Size([32, 8, 64, 64])
```

PyTorch Example 3

- More interestingly, we can add a stride to the convolution to increase our resolution!

```
convt = nn.ConvTranspose2d(in_channels=16,  
                           out_channels=8,  
                           kernel_size=5,  
                           stride=2,  
                           padding=2)
```

```
x = torch.randn(32, 16, 64, 64)  
y = convt(x)  
y.shape
```

```
torch.Size([32, 8, 127, 127])
```

PyTorch Example 4

- Output padding is another type of padding that adds an additional row and column to the output. Easy to mix it up with padding.

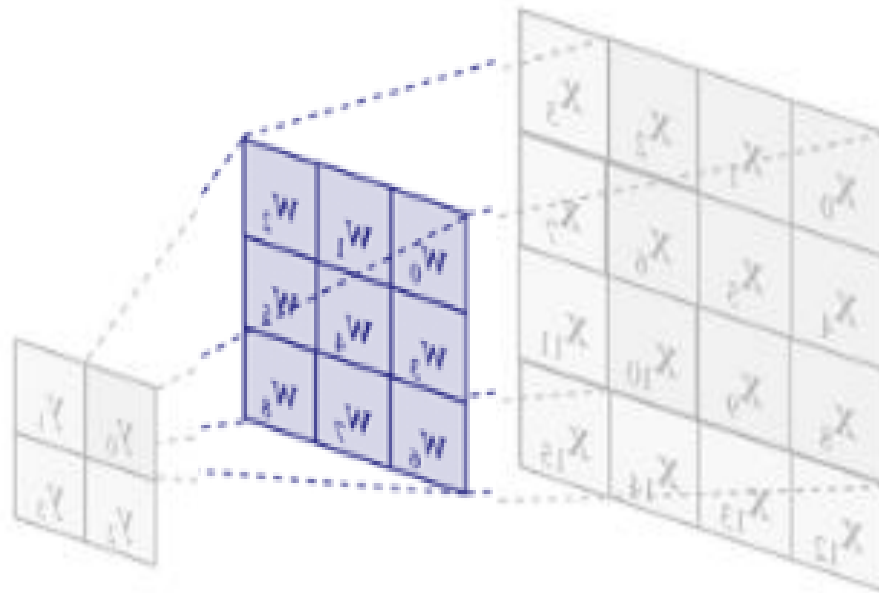
```
convt = nn.ConvTranspose2d(in_channels=16,  
                           out_channels=8,  
                           kernel_size=5,  
                           stride=2,  
                           padding=2,  
                           output_padding=1)
```

```
x = torch.randn(32, 16, 64, 64)  
y = convt(x)  
y.shape
```

```
torch.Size([32, 8, 128, 128])
```


Learning Kernel Weights

- The upsampling kernels can be learned just like normal convolutional kernels.
- Train weights to undo the convolutions from earlier



Convolutional Autoencoder

- Let's put everything together to build a convolutional autoencoder
- Just have to change the architecture, everything else should stay the same
- Can be difficult to verify that the input and output dimensions match
 - May require that you adjust output padding

PyTorch Implementation

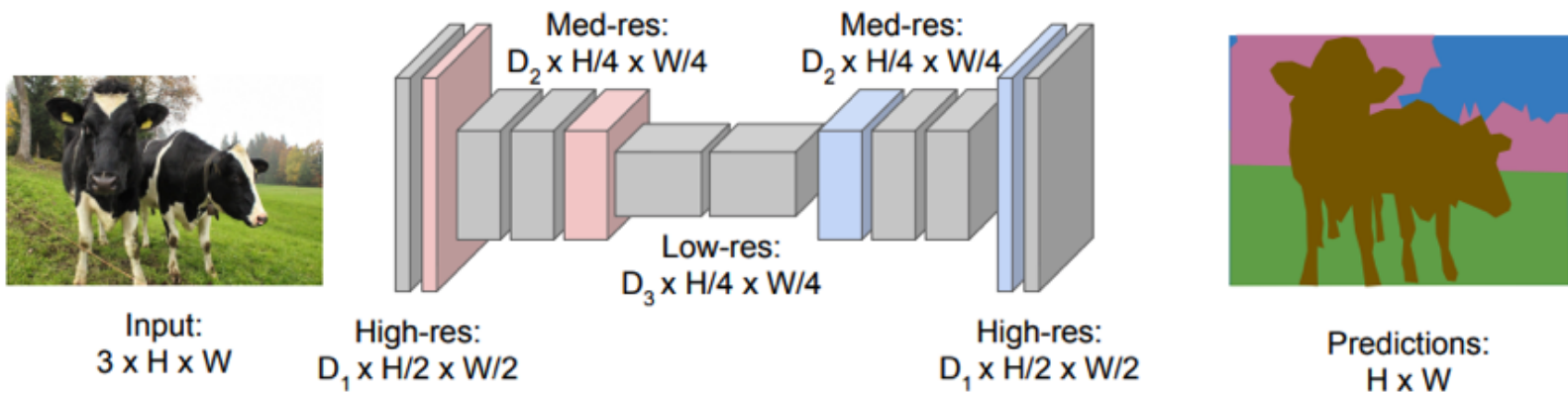
```
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 7)
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 7),
            nn.ReLU(),
            nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1,
                               output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1,
                               output_padding=1),
            nn.Sigmoid()
        )
```

PyTorch Implementation

```
def forward(self, x):  
    x = self.encoder(x)  
    x = self.decoder(x)  
    return x
```

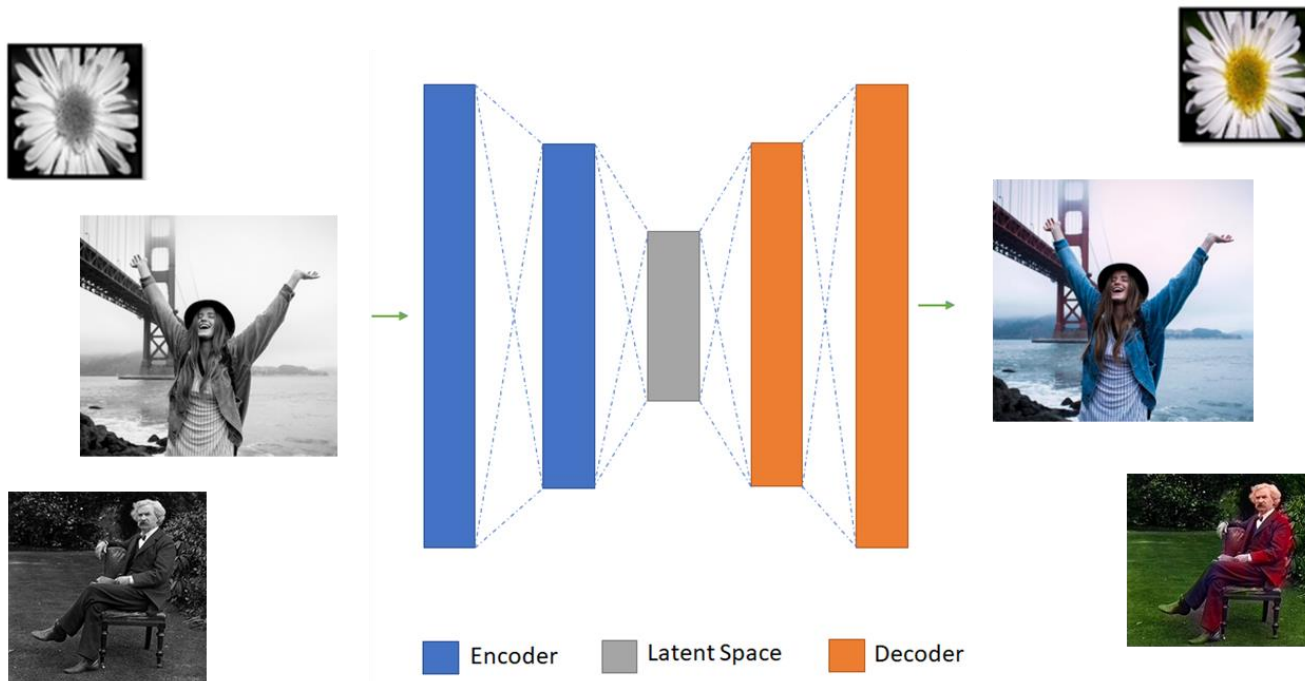
Application: Semantic Segmentation

➤ Q: How can we use an autoencoder for semantic segmentation?



Application: Image Colourization

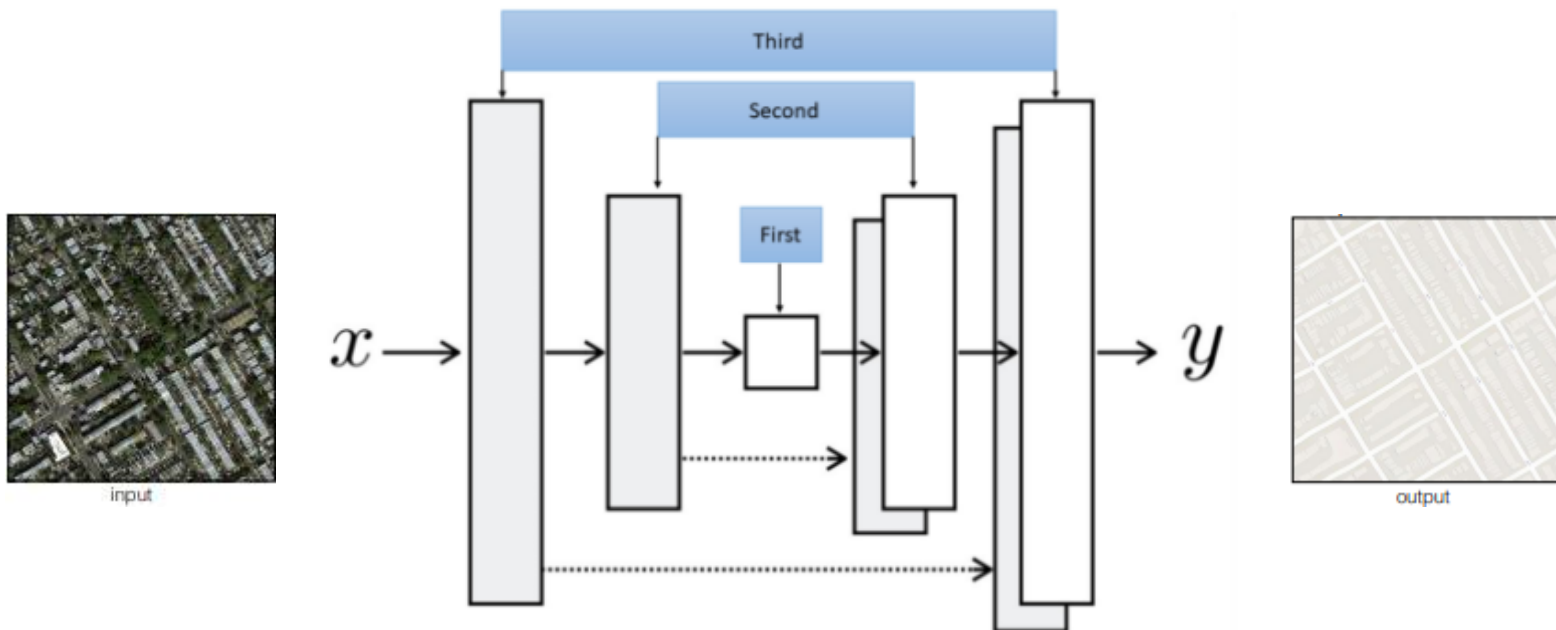
➤ Q: How can we use an autoencoder for image colourization?



U-Net Architecture

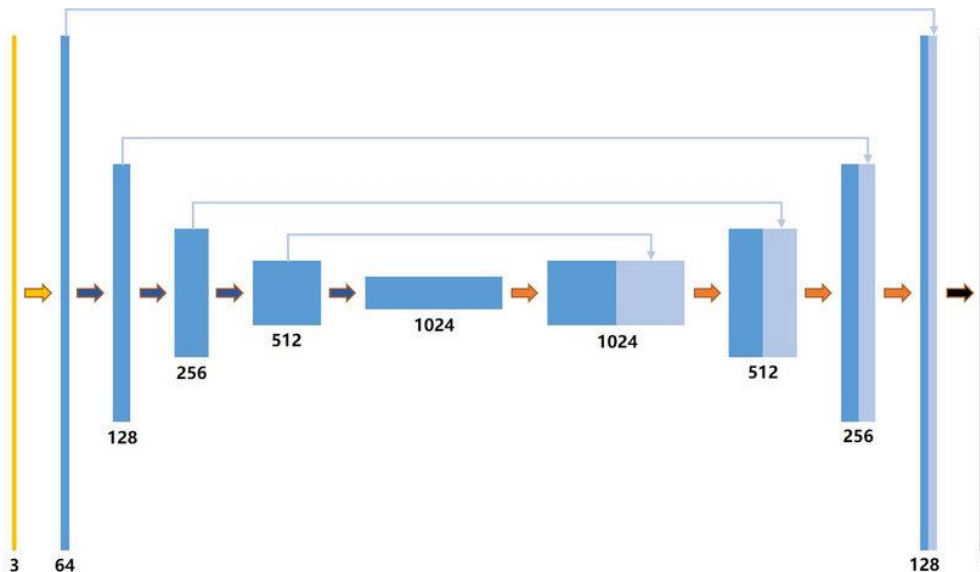
Skip Connections

- We've seen how skip connections can help improve CNNs, similar ideas can be applied to autoencoders.



Skip Connections

- Copy the output from encoder layer and concatenate as input to a decoder layer
- U-Net architecture is mirrored



Q: Why do skip connection work work?

Semi-Supervised Learning

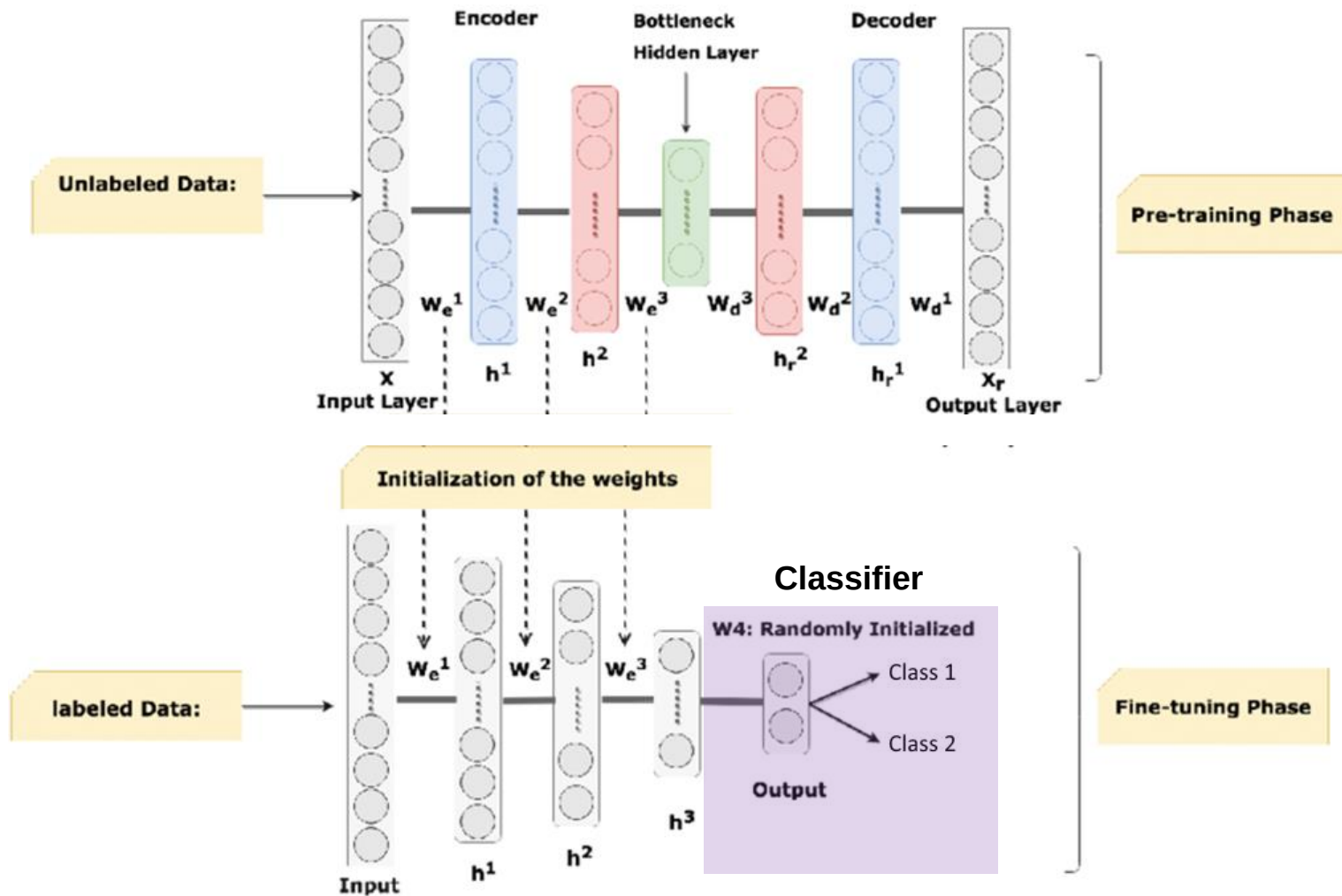
Unsupervised Pretraining

- Previously we discussed how **transfer learning** could use features obtained from ImageNet data to improve classification on other image tasks.
 - Assumption that the ImageNet data is similar in the new task.
 - If the new task is to detect new objects from similar images, then transfer learning makes sense.
- Autoencoders can achieve something similar by pretraining on large set of unlabeled data, same type of data, just missing labels

Semi-Supervised Learning

- This idea of Semi-Supervised learning was one of the motivators for the current Deep Learning wave that has reignited research in AI
 - Geoffrey Hinton with Deep Belief Networks and Restricted Boltzman Machines
 - Yoshua Bengio with Autoencoders
 - Yan LeCun with Self-Supervised learning

Semi-Supervised Learning



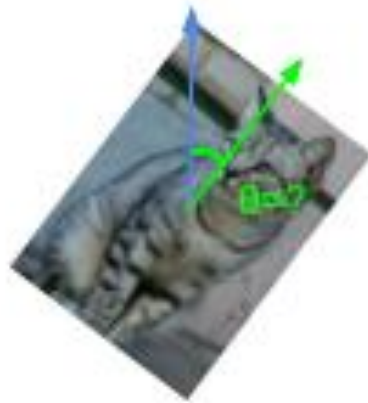
Self-Supervised Learning

- We've seen that we can perform semi-supervised learning by learning to reconstruct images.
- The **power of these models comes from not requiring labeled data** – it involves using what we now call **self-supervised learning**.

Self-Supervised Learning: Pretext Tasks



image completion



rotation prediction



"jigsaw puzzle"



colorization

- Solve the pretext task allow the model to learn good features.
- We can automatically generate labels for pretext tasks.

Self-Supervised Learning

Pretext task: predict rotations



90° rotation



270° rotation



180° rotation



0° rotation



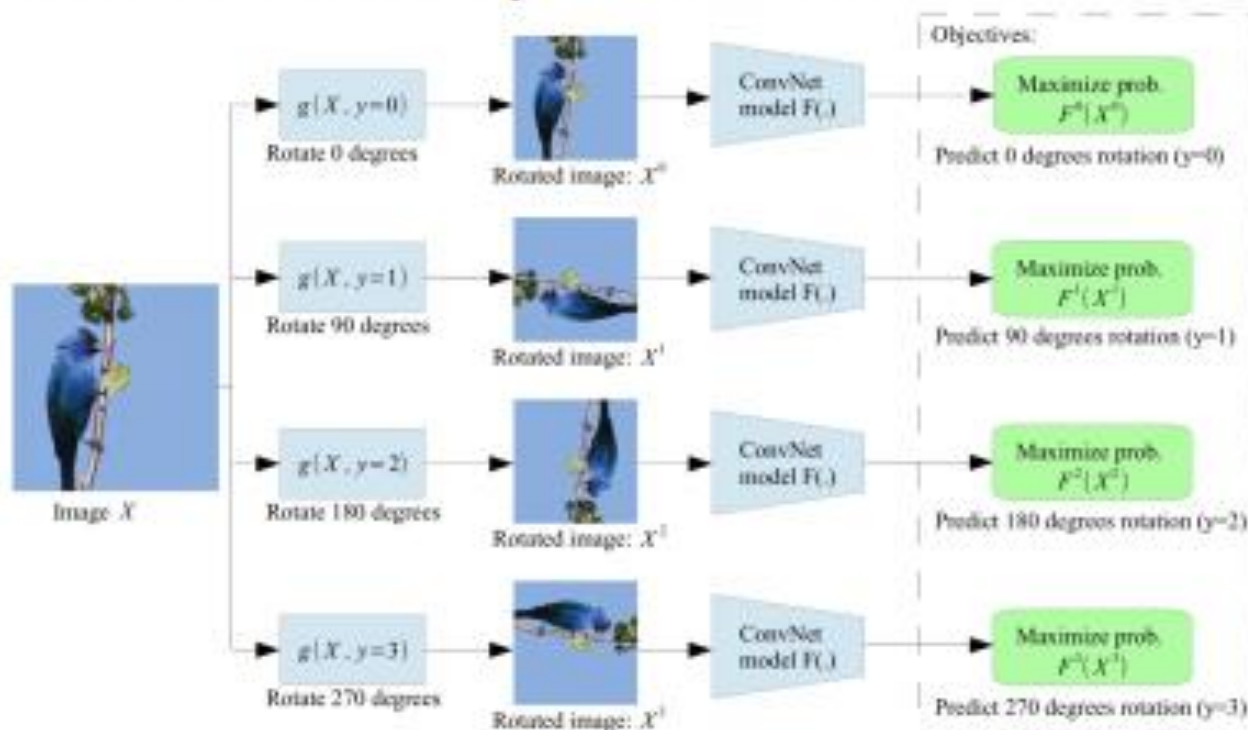
270° rotation

Hypothesis: a model could recognize the correct rotation of an object only if it has the “visual commonsense” of what the object should look like unperturbed.

(Image source: [Gidaris et al. 2018](#))

Self-Supervised Learning

Pretext task: predict rotations



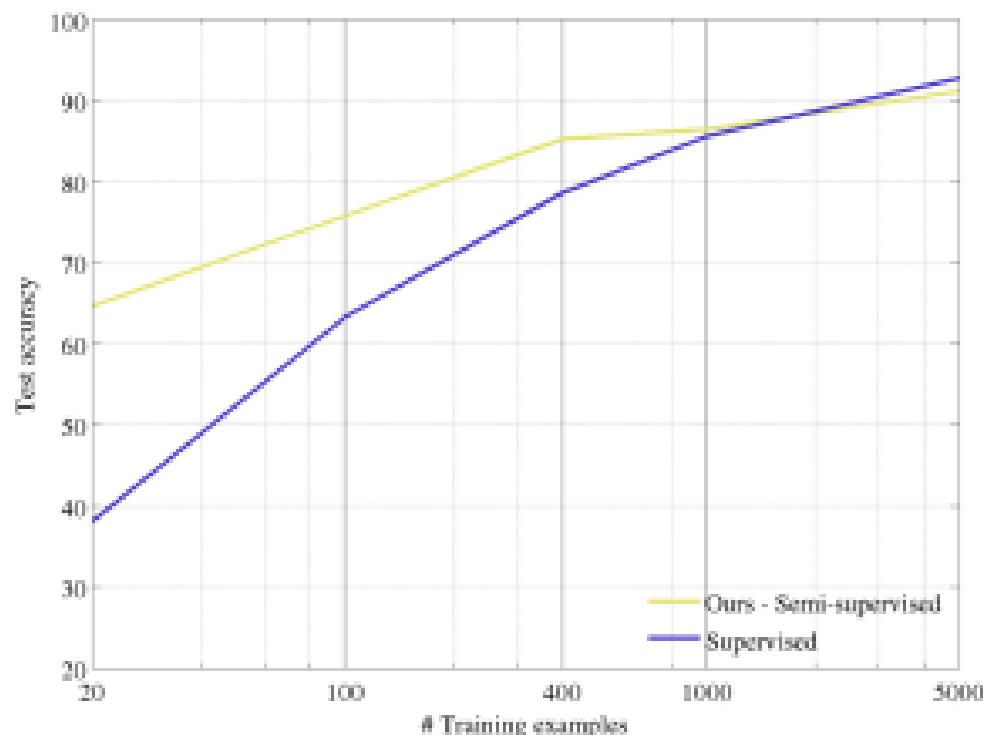
Self-supervised learning by rotating the entire input images.

The model learns to predict which rotation is applied (4-way classification)

(Image source: [Gidaris et al. 2018](#))

Self-Supervised Learning

Evaluation on semi-supervised learning



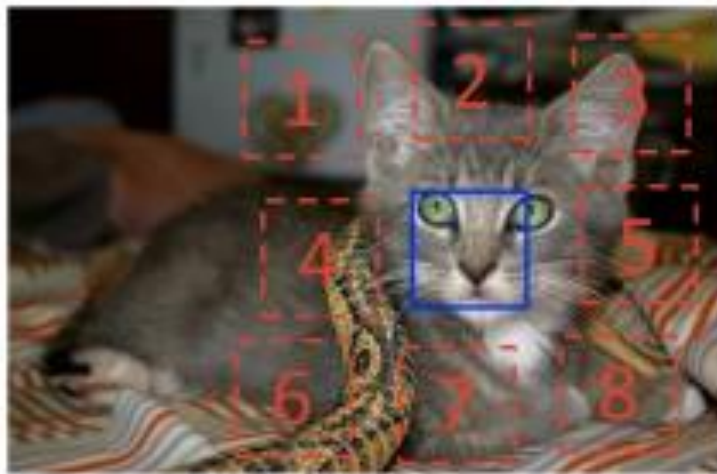
Self-supervised learning on
CIFAR10 (entire training set).

Freeze conv1 + conv2
Learn **conv3 + linear** layers
with subset of labeled
CIFAR10 data (classification).

(Image source: [Gidaris et al. 2018](#))

Self-Supervised Learning

Pretext task: predict relative patch locations



$$X = \left(\begin{array}{c} \text{cat face} \\ \text{cat ear} \end{array} \right); Y = 3$$

Example:



Question 1:



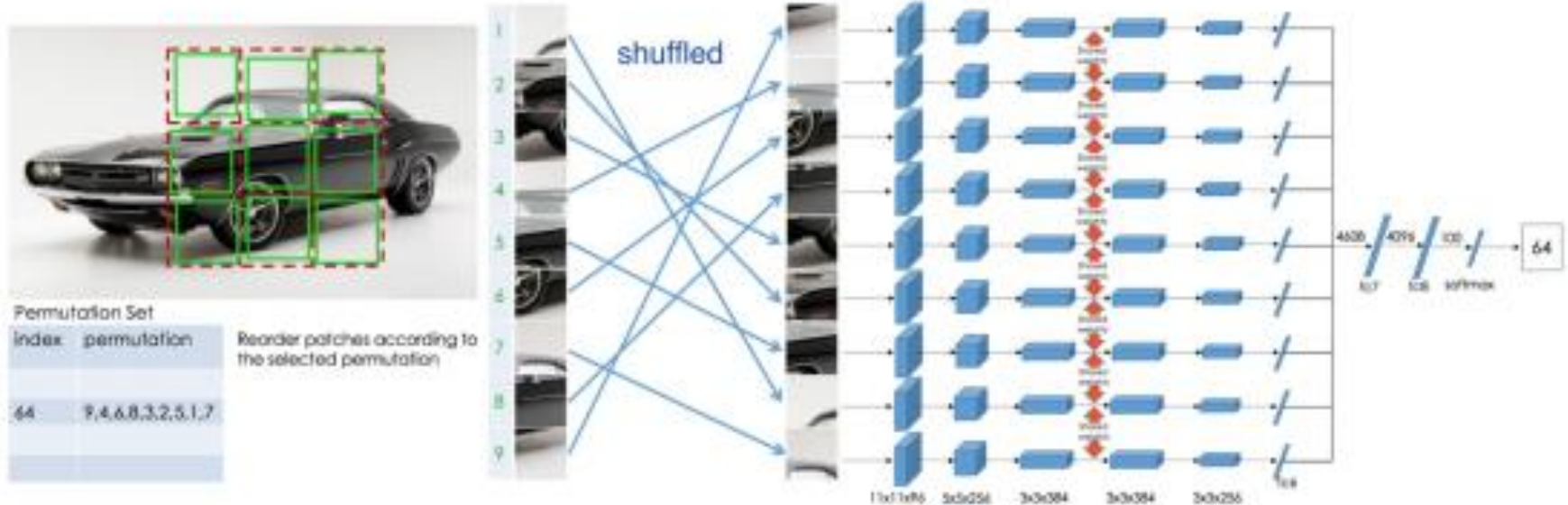
Question 2:



(Image source: [Doersch et al., 2015](#))

Self-Supervised Learning

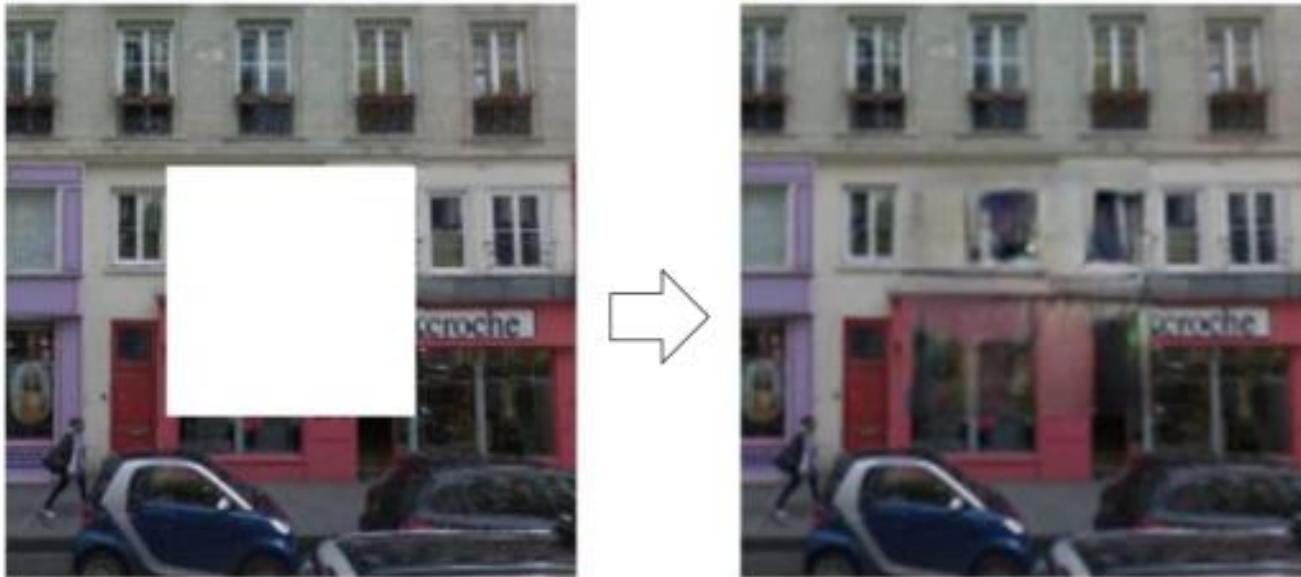
Pretext task: solving “jigsaw puzzles”



(Image source: [Noroozi & Favaro, 2016](#))

Self-Supervised Learning

Pretext task: predict missing pixels (inpainting)

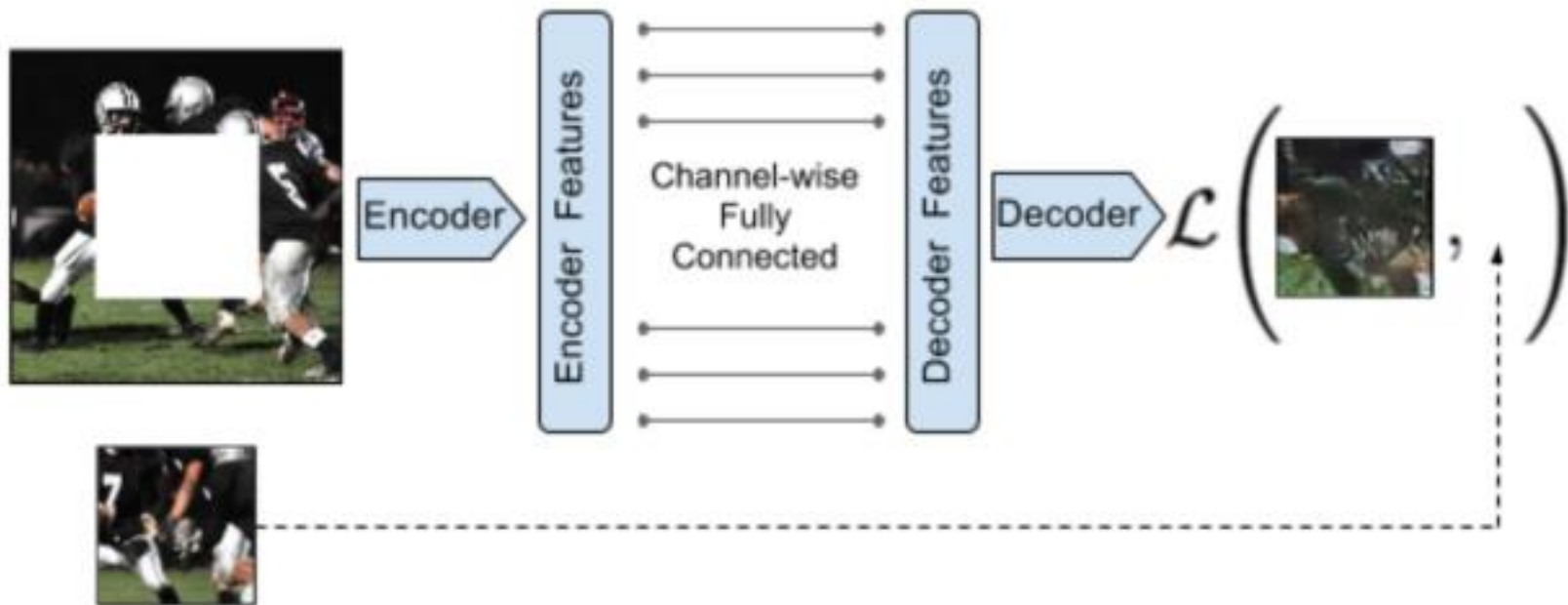


Context Encoders: Feature Learning by Inpainting (Pathak et al., 2016)

Source: [Pathak et al., 2016](#)

Self-Supervised Learning

Learning to inpaint by reconstruction



Learning to reconstruct the missing pixels

Source: [Pathak et al., 2016](#)

Self-Supervised Learning

Pretext task: image coloring



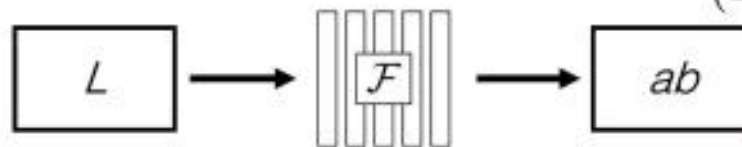
Grayscale image: L channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$



Concatenate (L, ab) channels

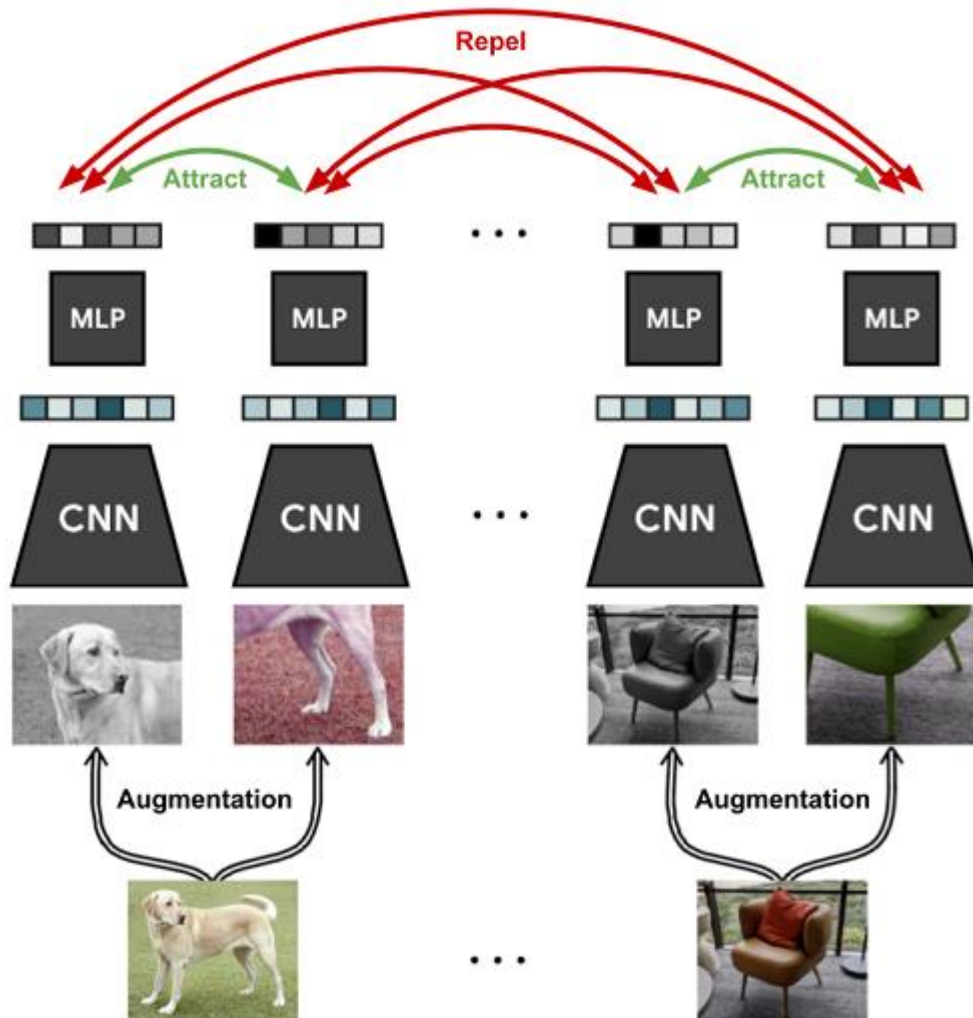
$$(\mathbf{X}, \hat{\mathbf{Y}})$$



Source: Richard Zhang / Phillip Isola

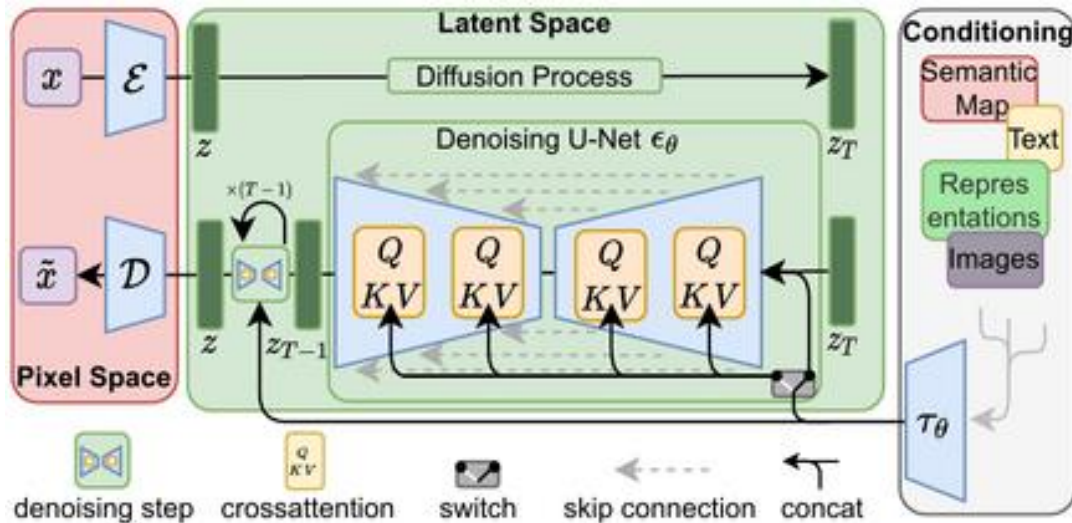
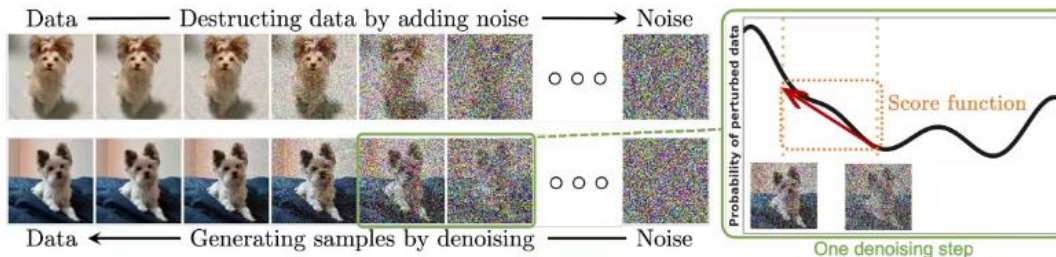
**Exciting Topics
(if time available)**

Contrastive Learning



- Representation Learning (learning good embeddings)
- Self-supervised learning through data augmentations

Diffusion Models



- U-Net architecture to predict noise and perform denoising
- Can be conditioned on text (more on this next time)

Summary

- Unsupervised Learning
- Embedding Space
 - reconstruction, denoising, self-supervised learning
 1. Generating New Data
 2. Feature Extraction
- Architectures for achieving the above
 - convolutional transpose, skip connection (U-Net)

Week 6 - Announcements

Assignment 2

- Assignment 2 Preparation
 - Pre_2A – CNN
 - Pre_2B – Transfer Learning
 - Ask questions in Support Session or on Piazza
- Assignment 2 – Hand Gesture Recognition
 - Create your own testing dataset (new data)
 - Dataset has errors – find and correct outliers or leave them and see how it affects performance
 - Large part of the evaluation depends on performance on new data
 - ~~Due Tuesday, Oct 7~~ **Saturday, Oct 11** at 11:00pm

Midterm Details

➤ When:

- Start Time: Wednesday, Oct 22, 1:10pm
- End Time: Wednesday, Oct 22, 2:40pm

➤ Duration:

- 1.5 hours

➤ Location:

- Sydney Smith Hall Room 2135

➤ Preparation:

- Midterm topics, sample problems and sample midterm posted on Quercus

Midterm Preparation

- A list of topics has been posted on Quercus
- Midterm Topics
 - Machine Learning Overview
 - Biological Motivations
 - Architectures of ANNs, CNNs, R-CNN, Autoencoders
 - Convolutions and Convolutional Transpose
 - Optimization and Hyperparameters,
 - Regularization
- Review Questions on Quercus
 - Sample midterm
 - Sample problem sets

Projects: Progress Report

- Progress reports due Tuesday, Nov 11th at 11:00pm
- Progress report is a check-in to show that you have made progress towards the completion of the project. Your team should have:
 - Obtained all or most of the data
 - Come up with a reasonable model
 - Produced at least one promising result
 - Provide a convincing argument for the feasibility of completing the project within the time available
- The progress report will be assessed out of 30 points based on: Data Collected, Results Achieved, and the clarity/readability of your Jupyter Notebook submission.

Next Time

Homework

- Review Pre3A, Pre3B Sample Code

Lecture Week 7

- Generative Adversarial Networks
- Adversarial Attacks

Q/A Support Session

- Assignment 3, Project Discussions, General Questions

Midterm (Wednesday, Oct 22 at 1:10pm – 2:40pm)

Reading Week (Oct 27 – Oct 31)